ROZDZIAŁ 4.

Uczenie modeli

Dotychczas traktowaliśmy modele uczenia maszynowego i ich algorytmy uczące niczym czarne skrzynki. Jeśli poświęciłaś/poświęciłeś czas na ćwiczenia umieszczone w poprzednich rozdziałach, dziwisz się zapewne, jak wiele możemy dokonać, nie znając podstaw działania modelu: zoptymalizować system regresyjny, usprawnić klasyfikator rozpoznawania cyfr, a nawet stworzyć od podstaw filtr spamu — wszystko to bez wiedzy na temat tworzących je mechanizmów. Faktycznie, w wielu przypadkach nie musimy znać szczegółów implementacji.

Jednak znajomość architektury modelu pomaga się w nim odnaleźć, dobrać właściwy algorytm uczący oraz zestaw hiperparametrów. Ułatwia nam to również usuwanie usterek z kodu, a także poprawia naszą skuteczność analizy błędów. Poza tym większość zagadnień poruszonych w tym rozdziale stanowi podwaliny zrozumienia, tworzenia i uczenia sieci neuronowych (którymi zajmiemy się w części II).

W niniejszym rozdziale zaczniemy od przyjrzenia się jednemu z najprostszych modeli — modelowi regresji liniowej. Omówimy dwa bardzo odmienne sposoby jego uczenia:

- Za pomocą „jawnego” wzoru bezpośrednio wyliczającego parametry, dzięki czemu model zostanie optymalnie wyuczony wobec zbioru uczącego (np. parametry minimalizujące funkcję kosztu dla zbioru uczącego).
- Za pomocą iteracyjnej metody optymalizacyjnej zwanej metodą gradientu prostego, dzięki której parametry modelu są stopniowo poprawiane w celu zminimalizowania funkcji kosztu wobec zestawu uczącego, co pozwala ostatecznie uzyskać te same wartości parametrów, co w przypadku pierwszej metody. Przeanalizujemy kilka odmian metody gradientu prostego, których będziemy bez przerwy używać podczas omawiania sieci neuronowych w części II, m.in. algorytmy: wsadowy gradientu prostego, schodzenia po gradiencie z minigrupami oraz stochastycznego spadku wzdłuż gradientu.

Następnie przejdziemy do regresji wielomianowej, czyli bardziej skomplikowanego modelu zdolnego do uczenia się wobec nieliniowych zbiorów danych. Model ten zawiera więcej parametrów niż algorytm regresji liniowej, przez co okazuje się bardziej podatny na przetrenowanie, dlatego skoncentrujemy się na sposobach wczesnego wykrywania tego problemu za pomocą krzywych uczenia, po czym zajmiemy się kilkoma technikami regularyzacji pozwalającymi zmniejszyć ryzyko przetrenowania.

Na koniec przyjrzymy się dwóm modelom powszechnie stosowanym w zadaniach klasyfikacji: regresji logistycznej i regresji softmax.



W rozdziale tym znajdziesz kilka równań matematycznych, wykorzystujących podstawową notację algebry liniowej i analizy matematycznej. Do ich zrozumienia wymagana jest podstawowa znajomość wektorów i macierzy, musisz znać pojęcia transpozycji, iloczynu skalarnego, macierzy odwrotnej i pochodnej cząstkowej. Jeśli te nazwy nic Ci nie mówią, zajrzyj do notatnika Jupyter zawierającego wprowadzenie do algebry liniowej i analizy matematycznej (*ftp://ftp.helion.pl/przyklady/troya.zip*). Jeśli nie znosisz matematyki, powinnaś/powinieneś tak czy siak przebrnąć przez ten rozdział, pomijając wszelkie wzory; mam nadzieję, że sam opis tekstowy wystarczy do zrozumienia większości pojęć.

Regresja liniowa

W rozdziale 1. przyjrzeliśmy się prostemu modelowi regresji liniowej prognozującemu satysfakcję z życia: *satysfakcja_z_życia* = θ_0+θ_1×*PKB_per_capita*.

Model ten stanowi jedynie funkcję liniową cechy wejściowej *PKB_per_capita*. Składowe θ_0 i θ_1 są parametrami tego modelu.

Ogólnie mówiąc, model liniowy otrzymuje prognozy poprzez obliczenie ważonej sumy cech wejściowych i stałej zwanej **punktem obciążenia** (ang. *bias term*) lub **punktem przecięcia** (ang. *intercept term*), co zostało ukazane w równaniu 4.1.

Równanie 4.1. Predykcja za pomocą modelu regresji liniowej

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

W tym równaniu:

- $\hat{y}$ — prognozowana wartość,
- n — liczba cech,
- x_1 — wartość i-tej cechy,
- θ_j — j-ty parametr modelu (w tym również punkt przecięcia θ_0 oraz wagi cech θ_1, θ_2, …, θ_n).

Możemy to zapisać w zwartej, zwektoryzowanej formie, tak jak w równaniu 4.2.

Równanie 4.2. Predykcja za pomocą modelu regresji liniowej (postać wektorowa)

$$\hat{y} = h_{\boldsymbol{\theta}}\left(\mathbf{x}\right) = \boldsymbol{\theta} \cdot \mathbf{x}$$

- $\boldsymbol{\theta}$ — wektor parametrów modelu, zawierający punkt obciążenia θ_0 i wagi cech od θ_1 do θ_n,
- $\boldsymbol{x}$ — wektor cech danego przykładu, zawierający cechy od x_0 do x_n, gdzie x_0 zawsze równa się 1,
- $\boldsymbol{\theta} \cdot \boldsymbol{x}$ — iloczyn skalarny wektorów $\boldsymbol{\theta}^T$ i $\boldsymbol{x}$, równy oczywiście wyrażeniu $\theta_0 x_0+\theta_1 x_1+\theta_2 x_2+\ldots+\theta_n x_n$,
- h_θ — funkcja hipotezy wykorzystująca parametry $\boldsymbol{\theta}$ modelu.



W uczeniu maszynowym wektory są często reprezentowane jako **wektory kolumnowe**, czyli tablice dwuwymiarowe zawierające jedną kolumnę. Jeżeli $\boldsymbol{\theta}$ i $\boldsymbol{x}$ są wektorami kolumnowymi, to predykcja przyjmuje postać $\hat{y} = \boldsymbol{\theta}^T\boldsymbol{x}$, gdzie $\boldsymbol{\theta}^T$ jest **transpozycją** macierzy $\boldsymbol{\theta}$ (wektorem wierszowym), natomiast operacja $\boldsymbol{\theta}^T\boldsymbol{x}$ to iloczyn macierzowy $\boldsymbol{\theta}^T$ i $\boldsymbol{x}$. Uzyskiwana prognoza jest oczywiście taka sama, jedyną różnicą jest jej reprezentowanie w jednoelementowej macierzy, a nie jako wartość skalarną. W tej książce będę używał tej notacji po to, aby unikać lawirowania pomiędzy iloczynem skalarnym a wektorowym.

No dobrze, tak wygląda model regresji liniowej, ale jak możemy go trenować? Przypominam, że uczenie modelu oznacza wyznaczenie jego parametrów w taki sposób, że będą optymalnie dopasowane do zbioru uczącego. W tym celu musimy najpierw zmierzyć, jak dobrze (lub jak źle) model jest dopasowany do tych danych. Z rozdziału 2. wiemy, że najpopularniejszą miarą wydajności dla modelu regresyjnego jest pierwiastek błędu średniokwadratowego (RMSE; równanie 2.1). Zatem w celu wyuczenia modelu regresji liniowej musimy określić wartość wektora $\boldsymbol{\theta}$ minimalizującą błąd RMSE. W praktyce znacznie łatwiej zminimalizować wartość błędu średniokwadratowego (ang. *mean square error* — MSE), przy czym uzyskujemy takie same rezultaty (ponieważ wartość minimalizująca funkcję minimalizuje również jej pierwiastek kwadratowy)[1].

Za pomocą równania 4.3 wyliczamy błąd MSE hipotezy h_θ wobec zestawu uczącego $\boldsymbol{X}$.

Równanie 4.3. Funkcja kosztu MSE dla modelu regresji liniowej

$$MSE\left(\mathbf{X}, h_{\boldsymbol{\theta}}\right) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Większość tych notacji została wyjaśniona w rozdziale 2. (zob. ramka „Notacje"). Jedyna różnica polega na tym, że zapisujemy h_θ zamiast h, aby podkreślić, że model zostaje sparametryzowany przez wektor $\boldsymbol{\theta}$. W celu uproszczenia będziemy zapisywać jedynie $MSE(\boldsymbol{\theta})$ zamiast $MSE(\boldsymbol{X}, h_\theta)$.

Równanie normalne

Aby określić wartość wektora $\boldsymbol{\theta}$ minimalizującą funkcję kosztu, używamy **jawnego rozwiązania** (ang. *closed-form solution*) — innymi słowy, wzoru matematycznego dającego bezpośredni wynik. Jest to tak zwane **równanie normalne** (rysunek 4.4).

Równanie 4.4. Równanie normalne

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^T \cdot X\right)^{-1} \cdot \mathbf{X}^T \cdot \mathbf{y}$$

[1] Często się zdarza, że algorytm uczący będzie próbował zoptymalizować inną funkcję niż miara wydajności używana do oceny ostatecznego modelu. Wynika to najczęściej z faktu, że dana funkcja jest łatwiejsza do wyliczenia, ponieważ zawiera przydatne własności różniczkowania, których brakuje metryce wydajności, lub ponieważ chcemy ograniczyć model na etapie uczenia, o czym się przekonasz, gdy będę omawiał regularyzację.

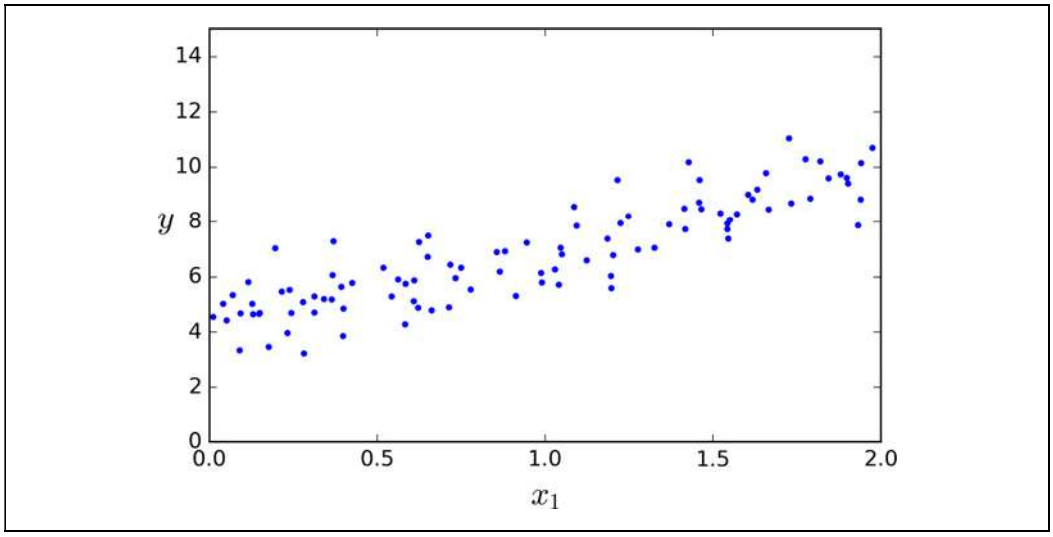
W tym równaniu:

- $\hat{\boldsymbol{\theta}}$ — wartość wektora $\boldsymbol{\theta}$ minimalizująca funkcję kosztu,
- $\boldsymbol{y}$ — wektor wartości docelowych od $y^{(1)}$ do $y^{(m)}$.

Wygenerujmy teraz dane przypominające z grubsza liniowe, aby przetestować omawiany wzór (rysunek 4.1):

```
import numpy as np

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```



Rysunek 4.1. *Losowo wygenerowany liniowy zbiór danych*

Obliczmy teraz wartość $\hat{\boldsymbol{\theta}}$ za pomocą równania normalnego. Skorzystamy z funkcji `inv()` dostępnej w module algebry liniowej Numpy (`np.linalg`), aby obliczyć macierz odwrotną, natomiast metoda `dot()` posłuży nam do przemnożenia macierzy:

```
X_b = np.c_[np.ones((100, 1)), X] # Dodaje x0 = 1 do każdej próbki
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

Dane wygenerowaliśmy za pomocą funkcji $y = 4+3x_1+szum\ gaussowski$. Sprawdźmy, co ten wzór wyliczył:

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

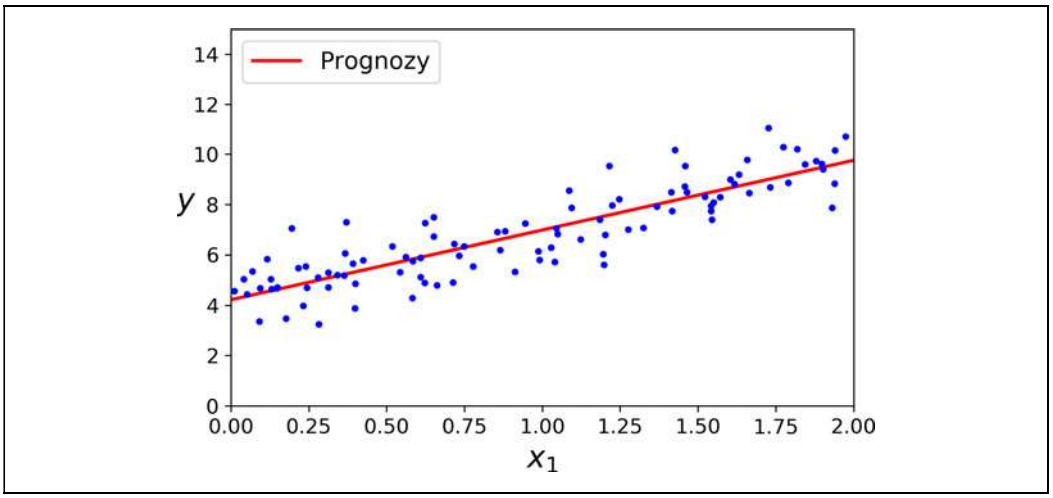
Oczekiwalibyśmy, że otrzymamy $\theta_0 = 4$ i $\theta_1 = 3$ zamiast $\theta_0 = 4{,}215$ i $\theta_1 = 2{,}770$. Wynik jest całkiem bliski optymalnemu, ale z powodu szumu nie jesteśmy w stanie odzyskać dokładnych parametrów pierwotnej funkcji.

Teraz za pomocą $\hat{\theta}$ możemy wyliczyć prognozy:

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # Dodaje x0 = 1 do każdej próbki
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

Narysujmy wykres prognoz wyliczonych przez ten model (rysunek 4.2):

```
plt.plot(X_new, y_predict, "r-")
plt.plot(X, y, "b.")
plt.axis([0, 2, 0, 15])
plt.show()
```



Rysunek 4.2. *Prognozy modelu regresji liniowej*

Przeprowadzenie regresji liniowej za pomocą Scikit-Learn jest proste[2]:

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

Klasa `LinearRegression` bazuje na funkcji `scipy.linalg.lstsq()` (skrót *lstsq* wywodzi się od angielskiej nazwy *least squares*, czyli „najmniejsze kwadraty"), którą możemy wywołać bezpośrednio:

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

[2] Zwróć uwagę, że moduł Scikit-Learn rozgranicza punkt przecięcia (`intercept_`) od wag cech (`coef_`).

Funkcja ta oblicza $\hat{\theta} = X^{+}y$, gdzie X^{+} jest pseudoodwrotnością macierzy X (mówiąc dokładniej, uogólnioną macierzą pseudoodwrotną Moore'a-Penrose'a). Możesz wykorzystać funkcję `np.linalg.pinv()` do bezpośredniego obliczenia macierzy pseudoodwrotnej:

```
>>> np.linalg.pinv(X_b).dot(y)
array([[4.21509616],
       [2.77011339]])
```

Sama w sobie macierz pseudoodwrotna jest obliczana za pomocą standardowej techniki faktoryzacji macierzy zwanej **rozkładem według wartości osobliwych** (ang. *Singular Value Decomposition* — SVD). Dzięki niej jesteśmy w stanie rozłożyć macierz zestawu danych uczących X na iloczyn trzech macierzy $U \Sigma V^T$ (więcej informacji znajdziesz w dokumentacji funkcji `numpy.linalg.svd()`). Macierz pseudoodwrotna jest obliczana jako $X^{+} = V\Sigma^{+}U^{T}$. W celu obliczenia macierzy Σ^{+} wyznacza w macierzy Σ zera w miejscu wszystkich wartości nieprzekraczających małej wartości progowej, następnie zastępuje wszystkie wartości niezerowe ich odwrotnościami, a na koniec transponuje tak uzyskaną macierz. Rozwiązanie to jest skuteczniejsze od rozwiązywania równania normalnego, a do tego nie sprawiają tu problemu skrajne przypadki: w rzeczy samej, równanie normalne może okazać się bezużyteczne, jeżeli macierz $X^{T}X$ jest odwracalna (tzn. osobliwa) tak, że $m < n$, lub w przypadku nadmiarowości niektórych cech, ale macierz pseudoodwrotna jest zawsze zdefiniowana.

Złożoność obliczeniowa

Równanie normalne wylicza odwrotność macierzy $X^{T}X$, dzięki czemu otrzymujemy macierz $(n+1)\times(n+1)$ (gdzie n oznacza liczbę cech). **Złożoność obliczeniowa** odwrócenia takiej macierzy wynosi zazwyczaj od około $O(n^{2,4})$ do $O(n^{3})$ (w zależności od implementacji). Inaczej mówiąc podwojenie liczby cech wydłuża czas obliczeń od około $2^{2,4} = 5,3$ do $2^{3} = 8$ razy.

Technika SVD wykorzystywana w klasie LinearRegression ma złożoność rzędu mniej więcej $O(n^{2})$. Jeżeli podwoisz liczbę cech, zwiększysz czas obliczeniowy w przybliżeniu czterokrotnie.



Szybkość zarówno równania normalnego, jak i algorytmu SVD znacznie się zmniejsza, gdy mamy do czynienia z dużą liczbą cech (np. 100 000). Dobra informacja jest taka, że obydwa rozwiązania są liniowe pod względem liczby przykładów tworzących zbiór danych uczących (złożoność obliczeniowa wynosi $O(m)$), co oznacza, że radzą sobie całkiem nieźle z dużymi zbiorami danych, przy założeniu, że zestawy te mieszczą się w pamięci.

Poza tym po wyuczeniu modelu regresji liniowej (za pomocą równania normalnego lub dowolnego innego algorytmu) prognozy są bardzo szybko wyliczane: złożoność obliczeniowa jest liniowa zarówno wobec liczby próbek, dla których chcemy uzyskać prognozy, jak i liczby cech. Innymi słowy, wyliczanie predykcji dla dwukrotnie większej liczby przykładów (lub cech) podwoi czas obliczeń.

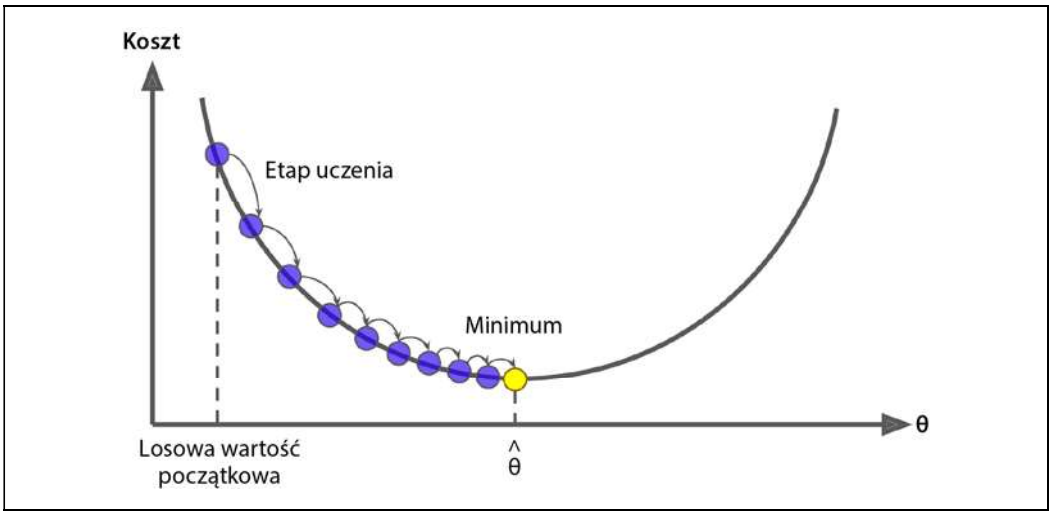
Teraz przyjrzymy się zgoła odmiennemu sposobowi uczenia modelu regresji liniowej, bardziej dostosowanemu do olbrzymiej liczby cech lub zbiorów danych niemieszczących się w pamięci.

Gradient prosty

Metoda **gradientu prostego** (ang. *gradient descent*) stanowi prosty algorytm optymalizacyjny służący do znajdowania optymalnych rozwiązań dla bardzo szerokiej gamy problemów. Ogólna koncepcja gradientu prostego polega na wielokrotnym poprawianiu wartości parametrów w celu zminimalizowania funkcji kosztu.

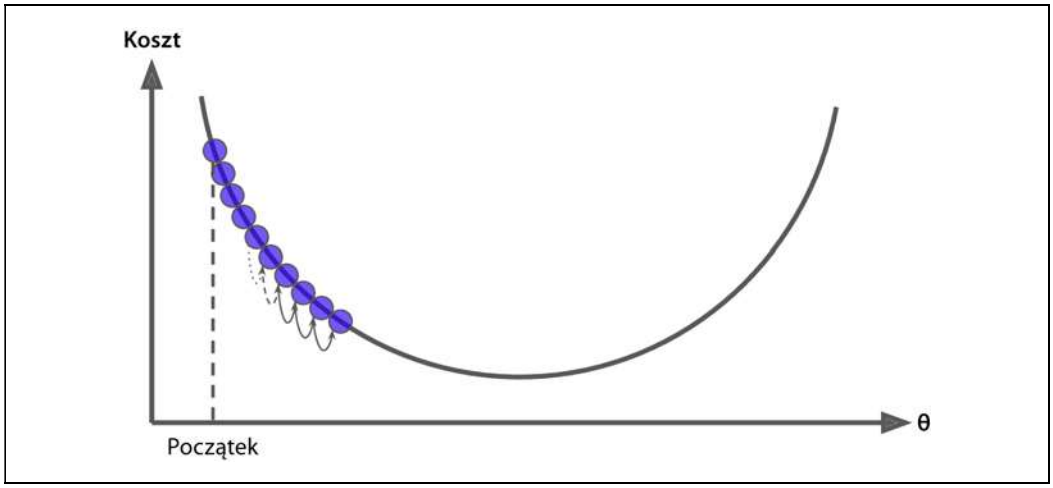
Załóżmy, że zabłądziliśmy w górach z powodu gęstej mgły i wyczuwamy jedynie nachylenie terenu pod stopami. Logicznym rozwiązaniem zejścia na dno doliny jest podążanie w dół po jak największej pochyłości. Dokładnie taki jest mechanizm działania gradientu prostego: algorytm mierzy lokalny gradient funkcji błędu w odniesieniu do wektora parametrów $\boldsymbol{\theta}$, a następnie podąża w kierunku malejącego gradientu. Po uzyskaniu wartości 0 docieramy do minimum funkcji!

Dokładniej mówiąc, zaczynamy od wypełnienia wektora $\boldsymbol{\theta}$ losowymi wartościami (jest to tak zwana **inicjacja losowa** — ang. *random initialization*). Następnie stopniowo doprecyzowujemy metodą małych kroków, gdzie w każdym kroku staramy się zredukować funkcję kosztu (np. błąd MSE), aż do momentu uzyskania **zbieżności** algorytmu z wartością minimalną tejże funkcji (rysunek 4.3).



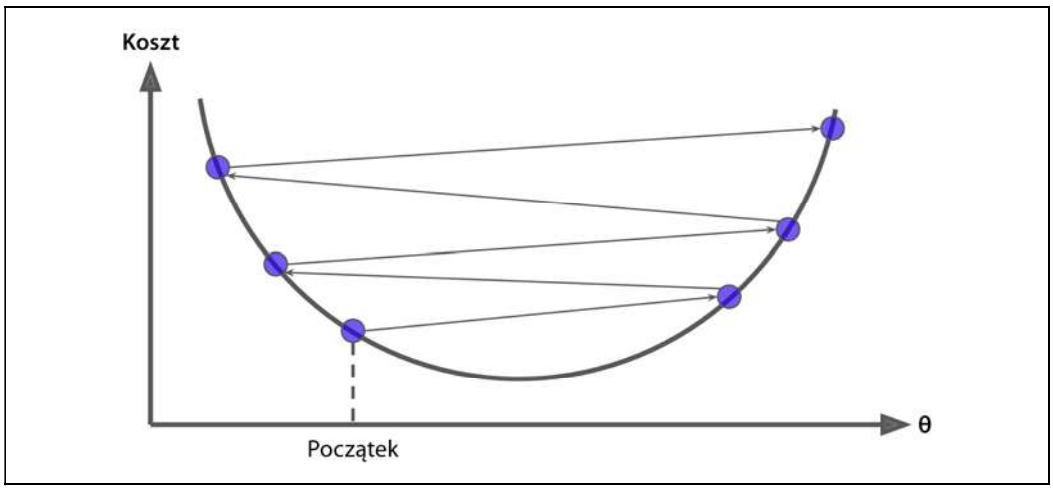
Rysunek 4.3. *Na powyższym schemacie gradientu prostego parametry modelu są inicjowane losowo i stopniowo są dostrajane w sposób minimalizujący funkcję kosztu; rozmiar kroku uczącego jest proporcjonalny do nachylenia funkcji kosztu, zatem w miarę zbliżania się do minimum kroki te stają się coraz mniejsze*

Ważnym elementem metody gradientu prostego jest rozmiar poszczególnych „kroków", określony za pomocą hiperparametru zwanego **współczynnikiem uczenia** (ang. *learning rate*). Jeżeli jego wartość jest zbyt mała, to algorytm będzie musiał wykonać wiele przebiegów w celu uzyskania zbieżności, co bywa czasochłonne (rysunek 4.4).



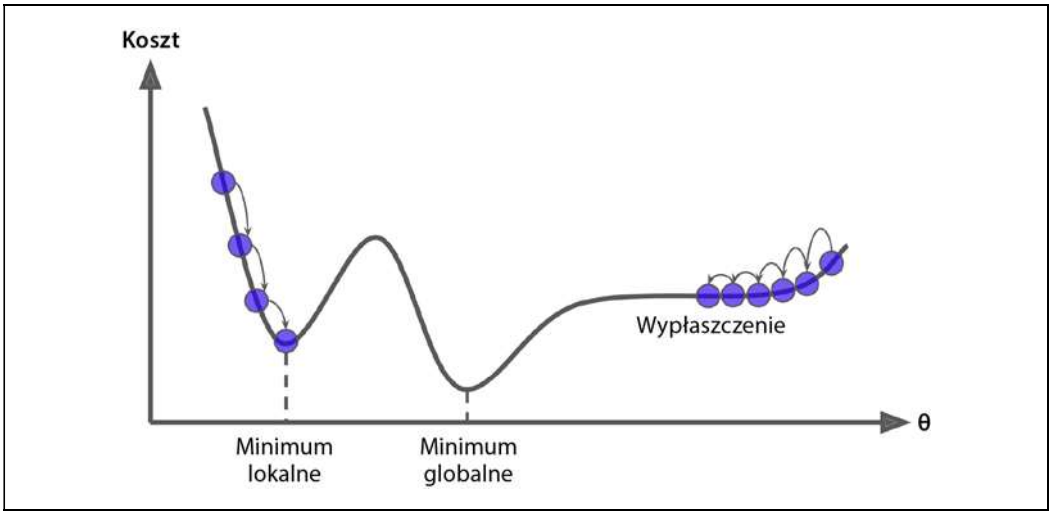
Rysunek 4.4. Zbyt mała wartość współczynnika uczenia

Z drugiej strony, przy zbyt dużej wartości współczynnika uczenia możesz przeoczyć dolinę i wylądować na przeciwległym wzniesieniu, być może nawet na większej wysokości niż wcześniej. W konsekwencji możemy otrzymać rozbieżny algorytm uzyskujący coraz większe wartości i niezdolny do znalezienia prawidłowego rozwiązania (rysunek 4.5).



Rysunek 4.5. Zbyt duża wartość współczynnika uczenia

Pamiętajmy również, że nie wszystkie funkcje kosztu mają taki ładny kształt. Mogą występować doliny i pagórki, wypłaszczenia, a także różnorodne inne nieregularności przebiegu funkcji, utrudniające uzyskanie zbieżności. Na rysunku 4.6 widzimy dwa główne wyzwania związane z gradientem prostym. Jeżeli inicjacja losowa rozpocznie się po lewej stronie wykresu, to algorytm stanie się zbieżny z **minimum lokalnym**, które nie jest tak dobre, jak **minimum globalne**. Z kolei jeśli rozpoczniemy proces poszukiwania minimum po prawej stronie wykresu, upłynie mnóstwo czasu, zanim algorytm przebrnie przez wypłaszczenie. Natomiast jeśli zatrzyma się zbyt wcześnie, nigdy nie osiągnie globalnego minimum.



Rysunek 4.6. Pułapki gradientu prostego

Na szczęście funkcja kosztu dla modelu regresji liniowej jest **wypukła**, co oznacza, że po wybraniu dwóch dowolnych punktów na jej przebiegu łączący je odcinek nigdy nie przetnie się z tą krzywą. Oznacza to, że nie istnieją lokalne minima, tylko jedno minimum globalne. Jest to również funkcja ciągła, której nachylenie nigdy nie zmienia się gwałtownie[3]. Obydwa te fakty mają bardzo ważny skutek: metoda gradientu prostego gwarantuje zbliżenie się na dowolną odległość do minimum globalnego (pod warunkiem że poczekamy wystarczająco długo, a współczynnik uczenia nie ma zbyt dużej wartości).

W rzeczywistości funkcja kosztu ma stożkowaty kształt, ale jeżeli cechy znajdują się w bardzo różniących się skalach, stożek ten może przyjąć wydłużoną postać. Na rysunku 4.7 widzimy metodę gradientu prostego stosowaną wobec zestawu uczącego, gdzie cechy 1. i 2. znajdują się w takiej samej skali (po lewej), oraz dla zbioru uczącego, w którym cecha 1. ma znacznie mniejsze wartości niż cecha 2. (po prawej)[4].

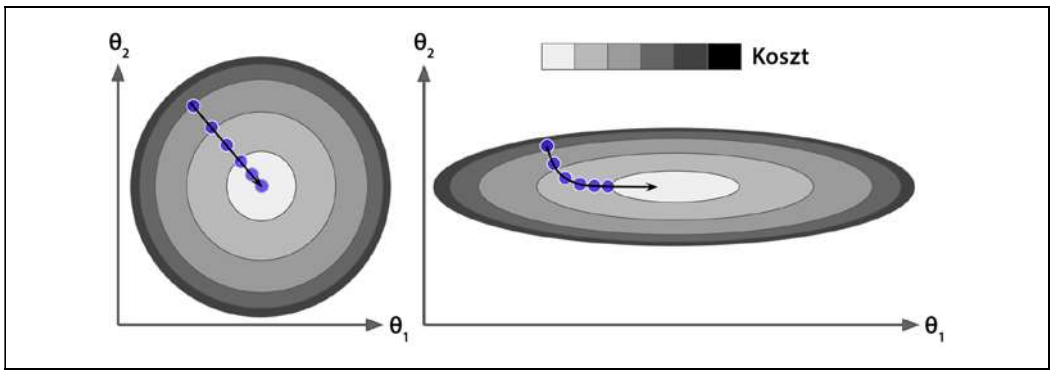
Jak widać na rysunku po lewej stronie, algorytm gradientu prostego dąży wprost do minimum, dzięki czemu szybko do niego dociera, natomiast na wykresie po prawej dąży najpierw niemal równolegle do osi y, po czym mnóstwo czasu spędza, poruszając się w niemal poziomym kierunku. W końcu osiągnie globalne minimum, ale minie wiele czasu.



Podczas korzystania z metody gradientu prostego należy umieścić wszystkie cechy w zbliżonej skali (np. za pomocą klasy `StandardScaler` modułu Scikit-Learn), gdyż w przeciwnym wypadku algorytm będzie potrzebował znacznie więcej czasu na osiągnięcie zbieżności.

[3] W ujęciu matematycznym ciągłość tej funkcji wynika z **warunku Lipschitza**.

[4] Cecha 1. ma mniejsze wartości, dlatego trzeba w większym stopniu zmienić wartość θ_1, aby wpłynąć na funkcję kosztu, dlatego wykres funkcji jest wydłużony wzdłuż osi x.



Rysunek 4.7. Metoda gradientu prostego przy skalowaniu cech (po lewej) i bez skalowania cech (po prawej)

Wykresy na rysunku 4.7 mówią nam również, że uczenie modelu oznacza poszukiwanie kombinacji parametrów minimalizujących funkcję kosztu (wobec zbioru uczącego). Są to poszukiwania w **przestrzeni parametrycznej** modelu: wraz z liczbą parametrów rośnie liczba wymiarów przestrzeni parametrycznej i tym trudniejsze stają się poszukiwania; szukanie igły w trzystuwymiarowym stogu siana jest znacznie trudniejsze niż w przestrzeni trójwymiarowej. Na szczęście w przypadku regresji liniowej funkcja kosztu jest wypukła, dlatego igła znajduje się na samym dnie stożka.

Wsadowy gradient prosty

Aby zaimplementować metodę gradientu prostego, musimy obliczyć gradient funkcji kosztu wobec każdego parametru θ_j modelu. Innymi słowy, musimy policzyć, jak bardzo funkcja kosztu ulegnie zmianie, jeżeli tylko troszeczkę zmodyfikujemy θ_j. Proces ten nosi nazwę wyliczania **pochodnej cząstkowej**. Można porównać go do pytania: „Jakie będzie nachylenie terenu pod moimi nogami, gdy się ustawię w kierunku wschodu?", następnie do zadania takiego samego pytania dla kierunku północnego itd. (oraz wielu innych kierunków, jeśli jesteś w stanie wyobrazić sobie Wszechświat składający się z większej liczby wymiarów niż standardowe trzy przestrzenne). Równanie 4.5 wylicza pochodną cząstkową funkcji kosztu dla parametru θ_j, co zapisujemy jako $\frac{\partial}{\partial \theta_j} MSE(\boldsymbol{\theta})$.

Równanie 4.5. Wzór na pochodną cząstkową funkcji kosztu

$$\frac{\partial}{\partial \theta_j} MSE(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

Zamiast pojedynczo wyliczać poszczególne pochodne cząstkowe, możesz wykorzystać równanie 4.6, aby policzyć je wszystkie naraz. Wektor gradientów, zapisywany jako $\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta})$, zawiera wszystkie pochodne cząstkowe funkcji kosztu (po jednej dla każdego parametru modelu).

Równanie 4.6. Wektor gradientów funkcji kosztu

$$\nabla_{\theta} MSE(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} MSE(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} MSE(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T \left(\mathbf{X}\theta - \mathbf{y} \right)$$



Zwróć uwagę, że powyższy wzór na każdym etapie gradientu prostego przeprowadza obliczenia wobec całego zbioru uczącego ***X***. Dlatego algorytm ten jest nazywany **wsadowym gradientem prostym** (ang. *batch gradient descent*): wykorzystuje on pełny zbiór danych uczących w każdym przebiegu (prawdopodobnie lepszą nazwą byłaby *pełny gradient prosty*). Wynika z tego, że jest on bardzo powolny w przypadku dużych zestawów danych (niedługo jednak poznamy znacznie szybsze algorytmy gradientu prostego). Jednak algorytm ten znakomicie dostosowuje się do liczby cech; uczenie modelu regresji liniowej za pomocą algorytmu gradientu prostego przebiega znacznie szybciej niż przy użyciu równania normalnego lub rozkładu SVD.

Po wyliczeniu wektora gradientów (będzie on wskazywał jakiś punkt na wzniesieniu funkcji) wystarczy skierować się w przeciwną stronę, aby zacząć schodzić w dół funkcji. W ujęciu matematycznym oznacza to odjęcie wartości $\nabla_{\theta} MSE(\boldsymbol{\theta})$ od θ. Właśnie tutaj przydaje się współczynnik uczenia[5]: przemnażamy go (η) przez wektor gradientów, aby określić długość kroku (równanie 4.7).

Równanie 4.7. Określanie kroku gradientu prostego

$$\boldsymbol{\theta}^{(\text{kolejny krok})} = \boldsymbol{\theta} - \eta \nabla_{\theta} MSE(\boldsymbol{\theta})$$

Zobaczmy, jak wygląda implementacja tego algorytmu:

```
eta = 0.1 # Współczynnik uczenia
n_iterations = 1000
m = 100

theta = np.random.randn(2,1) # Losowa inicjacja

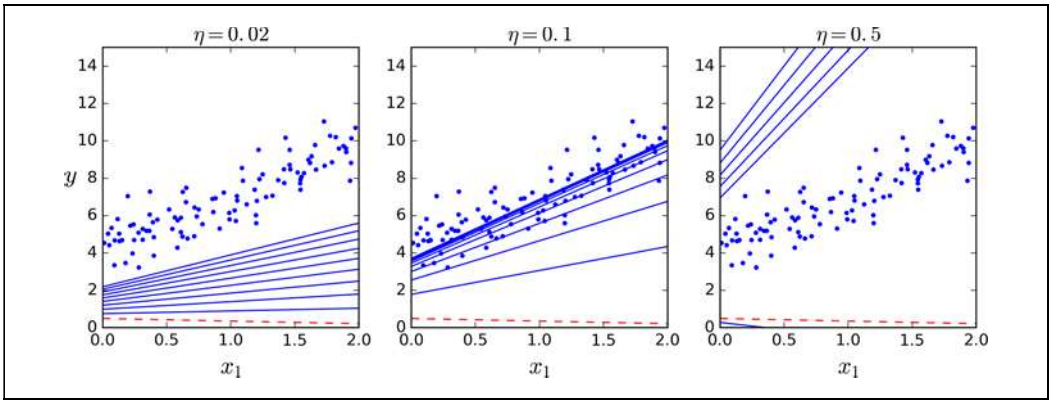
for iteration in range(n_iterations):
   gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
   theta = theta - eta * gradients
```

To nie było takie trudne! Sprawdźmy, jaką otrzymamy wartość współczynnika `theta`:

```
>>> theta
array([[ 4.21509616],
       [ 2.77011339]])
```

[5] Czyli η (eta) — siódma litera w alfabecie greckim.

Dokładnie taki sam rezultat uzyskaliśmy za pomocą równania normalnego! Algorytm gradientu prostego zadziałał znakomicie. A co by się stało, gdybyśmy użyli innej wartości współczynnika uczenia `eta`? Na rysunku 4.8 widzimy 10 pierwszych kroków gradientu prostego wykorzystujących trzy różne współczynniki uczenia (linia przerywana ukazuje początkowy etap).



Rysunek 4.8. *Różne współczynniki uczenia w algorytmie gradientu prostego*

Na lewym wykresie współczynnik uczenia ma zbyt małą wartość: algorytm ostatecznie znajdzie rozwiązanie, ale zajmie mu to mnóstwo czasu. Wartość współczynnika uczenia na środkowym wykresie prezentuje się całkiem nieźle: wystarczyło kilka przebiegów, aby algorytm uzyskał zbieżność. Z kolei na prawym wykresie wyznaczyliśmy zbyt dużą wartość współczynnika uczenia: algorytm staje się rozbieżny z funkcją, „porusza się" po całym układzie współrzędnych i w konsekwencji coraz bardziej oddala się od rozwiązania.

W celu znalezienia optymalnej wartości współczynnika uczenia możemy skorzystać z metody przeszukiwania siatki (opisanej w rozdziale 2.). Jednak możesz chcieć ograniczyć liczbę przebiegów, dzięki czemu metoda przeszukiwania siatki wyeliminuje modele, którym uzyskanie zbieżności zajmuje zbyt wiele czasu.

Zastanawiasz się pewnie, jak wyznaczyć liczbę przebiegów. Jeżeli będzie zbyt mała, algorytm nie zdąży zbliżyć się do optymalnego rozwiązania, natomiast przy zbyt dużej liczbie przebiegów będziesz marnować czas, a parametry modelu nie będą się już zmieniać. Prostym rozwiązaniem jest wyznaczenie dużej liczby przebiegów, a następnie przerwanie działania algorytmu, gdy wartość wektora gradientów stanie się bardzo mała — tzn. wtedy, gdy stanie się mniejsza od wartości parametru ε (tzw. **tolerancji**) — gdyż tak się dzieje, kiedy algorytm gradientu prostego osiąga (niemal) minimum.

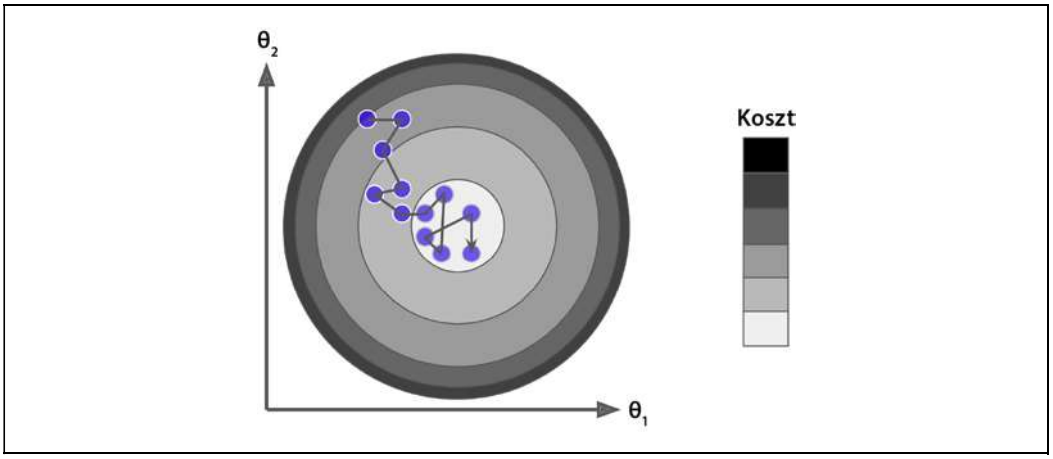
Szybkość uzyskania zbieżności

W przypadku wypukłych funkcji kosztu, których nachylenie nie zmienia się gwałtownie (czyli np. funkcji kosztu MSE), algorytm wsadowego gradientu prostego o stałym współczynniku uczenia w końcu osiągnie optymalne rozwiązanie, musimy jednak trochę na to poczekać: uzyskanie optimum w zakresie ε może zająć $O(1/\varepsilon)$ przebiegów w zależności od kształtu funkcji kosztu. Jeżeli podzielimy wartość tolerancji przez 10 w celu uzyskania precyzyjnego rozwiązania, to algorytm może poświęcić na to jakieś dziesięć razy więcej czasu.

Stochastyczny spadek wzdłuż gradientu

Główny problem algorytmu wsadowego gradientu prostego polega na tym, że wykorzystuje on pełny zbiór danych uczących do obliczania gradientu podczas każdego przebiegu, przez co wraz ze wzrostem rozmiaru zbioru uczącego spada jego szybkość. Jego przeciwieństwo stanowi algorytm **stochastycznego spadku wzdłuż gradientu** (ang. *stochastic gradient descent* — SGD), w którym podczas każdego przebiegu jest dobierana losowa próbka ucząca, za pomocą której są wyliczane gradienty. Oczywiście przetwarzanie pojedynczych przykładów sprawia, że algorytm ten jest znacznie szybszy, gdyż w każdym przebiegu wykorzystuje niewiele danych. Dzięki temu możemy go również używać wobec dużych zestawów danych, ponieważ podczas każdego przebiegu jest przechowywany w pamięci tylko jeden przykład (algorytm SGD możemy zaimplementować również w postaci modelu pozakorowego; zob. rozdział 1.).

Z drugiej strony, w wyniku stochastycznej (tj. losowej) natury algorytm ten jest znacznie bardziej „chaotyczny" od jego wsadowego odpowiednika: funkcja kosztu nie schodzi tu łagodnie w kierunku minimum, lecz przeskakuje raz wyżej, raz niżej i dąży do minimum jedynie po uśrednieniu wyników. Po pewnym czasie w końcu trafi w okolicę bliską minimum, ale nie zatrzyma się i ciągle będzie „skakać" (rysunek 4.9). Zatem po zatrzymaniu algorytmu uzyskane wartości będą dobre, ale nie idealnie optymalne.



Rysunek 4.9. *Dzięki algorytmowi stochastycznego spadku wzdłuż gradientu każdy krok uczący staje się znacznie szybszy, ale jednocześnie bardziej losowy w stosunku do algorytmu wsadowego gradientu prostego*

W przypadku bardzo nieregularnych funkcji (rysunek 4.6) kosztu rozwiązanie to może pomóc w ucieczce z minimum lokalnego, zatem algorytm stochastycznego spadku wzdłuż gradientu ma większe szanse na znalezienie minimum globalnego niż wsadowy gradient prosty.

Zatem losowość przydaje się do ucieczki z minimów lokalnych, ale jednocześnie nie pozwala osiągnąć najniższego punktu funkcji kosztu. Jednym z rozwiązań tego problemu jest stopniowe zmniejszanie współczynnika uczenia. Początkowe kroki są duże (przyśpieszamy w ten sposób działanie algorytmu i wydostajemy się z minimów lokalnych), a następnie stają się coraz krótsze, dzięki czemu algorytm pozostaje w okolicach minimum globalnego. Proces ten jest podobny do **symulowanego wyżarzania** (ang. *simulated annealing*), algorytmu przypominającego proces wyżarzania w metalurgii, który

polega na tym, że roztopiony metal jest stopniowo chłodzony. Funkcja określająca współczynnik uczenia w każdym przebiegu jest nazywana **harmonogramem uczenia** (ang. *learning schedule*). Jeżeli zbyt szybko zredukujemy ten współczynnik, możemy utknąć w minimum lokalnym, a nawet pozostać gdzieś w połowie stoku. Z kolei jeżeli harmonogram uczenia będzie malał zbyt powoli, możemy przez długi czas pomijać minimum globalne i uzyskać nieoptymalne rozwiązanie, jeśli zbyt wcześnie zakończymy działanie algorytmu.

Za pomocą poniższego kodu implementujemy algorytm stochastycznego spadku wzdłuż gradientu wykorzystujący prosty harmonogram uczenia:

```
n_epochs = 50
t0, t1 = 5, 50 # Hiperparametry harmonogramu uczenia

def learning_schedule(t):
   return t0 / (t + t1)

theta = np.random.randn(2,1) # Losowa inicjacja

for epoch in range(n_epochs):
   for i in range(m):
      random_index = np.random.randint(m)
      xi = X_b[random_index:random_index+1]
      yi = y[random_index:random_index+1]
      gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
      eta = learning_schedule(epoch * m + i)
      theta = theta - eta * gradients
```

Zgodnie z konwencją nasz algorytm wykonuje *m* przebiegów; każdy przebieg jest nazywany **epoką**. Wcześniej używany algorytm wsadowego gradientu prostego wykonywał tysiąc przebiegów wobec pełnego zbioru danych uczących, natomiast tutaj uzyskujemy całkiem niezły wynik już po 50 epokach:

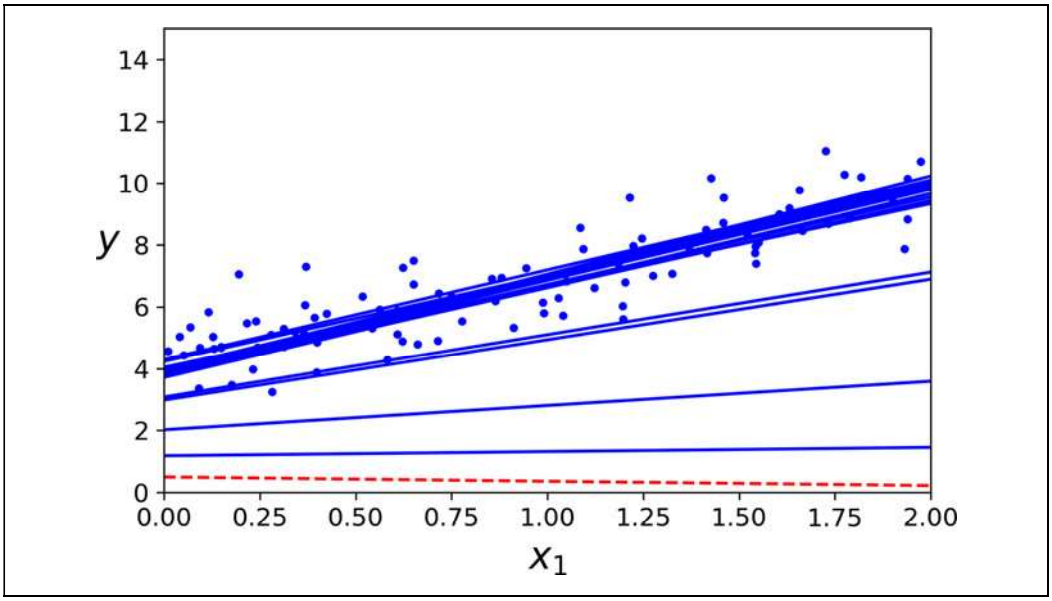
```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

Na rysunku 4.10 widzimy 20 pierwszych etapów uczenia (zwróć uwagę, jak bardzo są one nieregularne).

Zwróć uwagę, że z powodu losowości te same próbki mogą być kilkukrotnie pobierane w danej epoce, natomiast niektóre mogą wcale nie być użyte. Jeżeli chcesz mieć pewność, że zostaną wykorzystane różne dane w każdej epoce, dobrym rozwiązaniem jest przetasowanie wszystkich danych uczących (tak, aby cechy wejściowe i etykiety były wspólnie tasowane), następnie użycie każdego przykładu w danej epoce, przetasowanie zbioru danych przez kolejną epoką itd. Metoda ta jednak spowalnia osiągnięcie zbieżności przez algorytm.



Podczas korzystania z algorytmu stochastycznego spadku wzdłuż gradientu przykłady uczące muszą być niezależne i o tym samym rozkładzie (ang. *independent and identically distributed* — IIN), dzięki czemu średnio parametry będą dążyły ku optimum globalnemu. Łatwo to osiągnąć, tasując przykłady w trakcie uczenia (tzn. dobierając losowo każdy przykład lub tasując zestaw uczący na początku każdej epoki). Jeżeli nie przetasujesz przykładów (np. dane są posortowane zgodnie z etykietami), to algorytm SGD przeprowadzi optymalizację dla jednej etykiety, następnie dla następnej itd., przez co nie osiągnie minimum globalnego



Rysunek 4.10. *Dwadzieścia pierwszych przebiegów algorytmu stochastycznego spadku wzdłuż gradientu*

Aby wyuczyć model regresji liniowej za pomocą algorytmu SGD w module Scikit-Learn, możesz wykorzystać klasę `SGDRegressor`, która domyślnie wykonuje optymalizację wobec funkcji kosztu najmniejszych kwadratów. W poniższym kodzie algorytm będzie działał maksymalnie przez 1000 epok lub do momentu spadku funkcji straty poniżej 0,001 w trakcie jednej epoki (`max_iter=1000, tol=1e-3`). Początkowa wartość współczynnika uczenia wynosi 0,1 (`eta0=0.1`), oraz używamy domyślnego harmonogramu uczenia (różni się on od zastosowanego wcześniej harmonogramu). Na koniec nie korzystamy z żadnej formy regularyzacji (`penalty=None`; wkrótce wrócimy do tego zagadnienia):

```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1)
sgd_reg.fit(X, y.ravel())
```

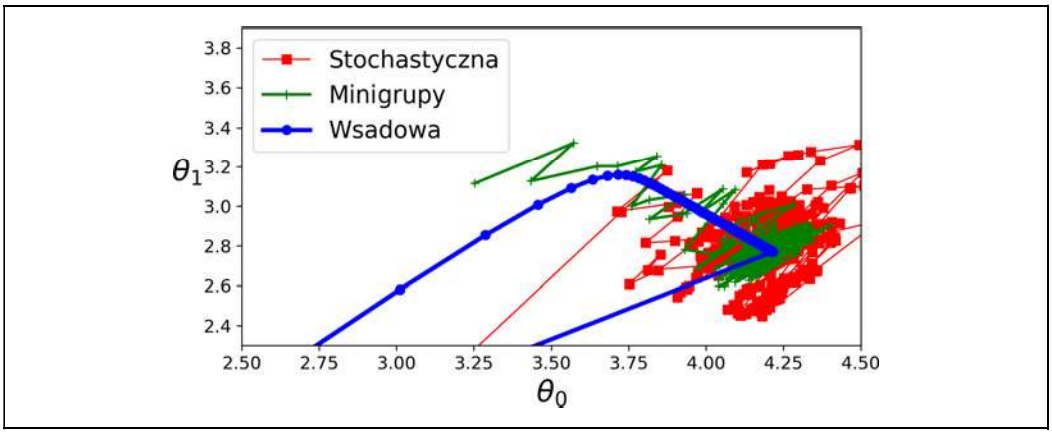
Również tym razem wynik będzie dość zbliżony do uzyskanego za pomocą równania normalnego:

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.24365286]), array([2.8250878]))
```

Schodzenie po gradiencie z minigrupami

Ostatnim rodzajem algorytmu gradientu prostego, jakim się zajmiemy, jest metoda **schodzenia po gradiencie z minigrupami** (ang. *mini-batch gradient descent*). Łatwo go zrozumieć, gdy znamy już pojęcia wsadowego gradientu prostego i stochastycznego spadku wzdłuż gradientu: tutaj w każdym przebiegu nie obliczamy gradientu na podstawie całego zbioru danych (metoda wsadowa) ani pojedynczych próbek (stochastyczny spadek), lecz przy użyciu niewielkich grup losowo dobieranych danych, tzw. **minigrup** lub **minipaczek** (ang. *mini-batches*). W porównaniu do stochastycznego spadku wzdłuż gradientu główną zaletą tej metody jest znaczne przyśpieszenie sprzętowej optymalizacji operacji na macierzach, zwłaszcza jeśli korzystamy z procesorów graficznych.

Przebieg algorytmu w przestrzeni parametrycznej jest mniej chaotyczny niż metody SGD, zwłaszcza w przypadku dość dużych minigrup. W wyniku tego dotrze on nieco bliżej minimum od algorytmu stochastycznego. Może jednak mu być trudniej wydostawać się z minimów lokalnych (w przypadku problemów generowanych przez lokalne minima, w przeciwieństwie do modelu regresji liniowej). Na rysunku 4.11 widzimy przebiegi trzech algorytmów gradientu prostego w przestrzeni parametrycznej na etapie uczenia. Wszystkie trzy docierają w okolicę minimum, ale algorytm wsadowego gradientu prostego praktycznie tam się zatrzymuje, natomiast metody stochastyczna i minigrup „błąkają się" dalej. Pamiętaj jednak, że metoda wsadowa potrzebuje znacznie więcej czasu na każdą epokę, a pozostałe dwa algorytmy również są w stanie osiągnąć minimum przy wprowadzeniu odpowiedniego harmonogramu uczenia.



Rysunek 4.11. Ścieżki algorytmów gradientu prostego w przestrzeni parametrycznej

Porównajmy omówione algorytmy, użyte przez nas do uczenia modelu regresji liniowej[6] (przypominam, że m oznacza liczbę przykładów uczących, natomiast n definiuje liczbę cech) (tabela 4.1).

Tabela 4.1. Porównanie algorytmów uczących model regresji liniowej

Algorytm	Duża wartość m	Uczenie pozakorowe	Duża wartość n	Hiper-parametry	Wymagane skalowanie	Scikit-Learn
Równanie normalne	Szybko	Nie	Powoli	0	Nie	Brak
SVD	Szybko	Nie	Powoli	0	Nie	`Linear ↪Regression`
Wsadowy gradient prosty	Powoli	Nie	Szybko	2	Tak	`SGD ↪Regressor`
Stochastyczny spadek wzdłuż gradientu	Szybko	Tak	Szybko	≥2	Tak	`SGD ↪Regressor`
Schodzenie po gradiencie z minigrupami	Szybko	Tak	Szybko	≥2	Tak	`SGD ↪Regressor`

[6] Równanie normalne może być stosowane wyłącznie wobec modelu regresji liniowej, natomiast, jak się niebawem przekonamy, metody gradientu prostego są używane do trenowania również wielu innych modeli.



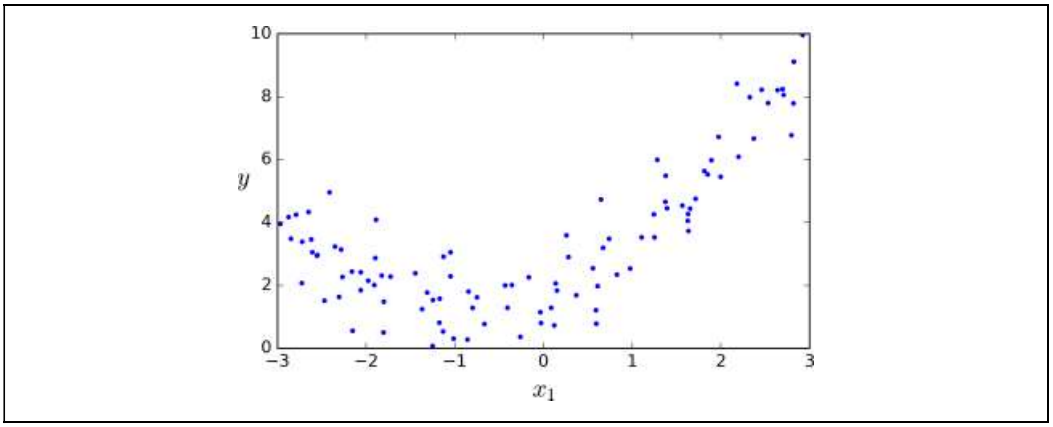
Różnica po wytrenowaniu tymi algorytmami jest niemal niezauważalna: za pomocą każdego z nich uzyskujemy niemal identyczny model, a prognozy są uzyskiwane prawie w taki sam sposób.

Regresja wielomianowa

Co w przypadku, jeśli Twoje dane nie układają się liniowo na wykresie? Co ciekawe, jesteśmy w stanie wytrenować model liniowy do rozpoznawania nieliniowych danych. Najprostszym sposobem jest dodanie potęg każdej cechy w postaci nowej cechy, a następnie wytrenowanie modelu wobec tak rozszerzonego zestawu cech. Technikę tę nazywamy **regresją wielomianową** (ang. *polynomial regression*).

Przeanalizujmy to na przykładzie. Najpierw wygenerujmy jakieś nieliniowe dane przy użyciu prostego **równania kwadratowego**[7] (oraz dodatkowego szumu) (rysunek 4.12):

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```



Rysunek 4.12. Wygenerowany zbiór nieliniowych i zaszumionych danych

Wyraźnie widać, że linia prosta nigdy nie będzie dobrze dopasowana do tych danych. Skorzystamy zatem z klasy `PolynomialFeatures` modułu Scikit-Learn, aby przekształcić zbiór danych uczących, i dodamy kwadrat (wielomian drugiego stopnia) każdej cechy jako nową cechę (w naszym przykładzie będzie dodana tylko jedna cecha):

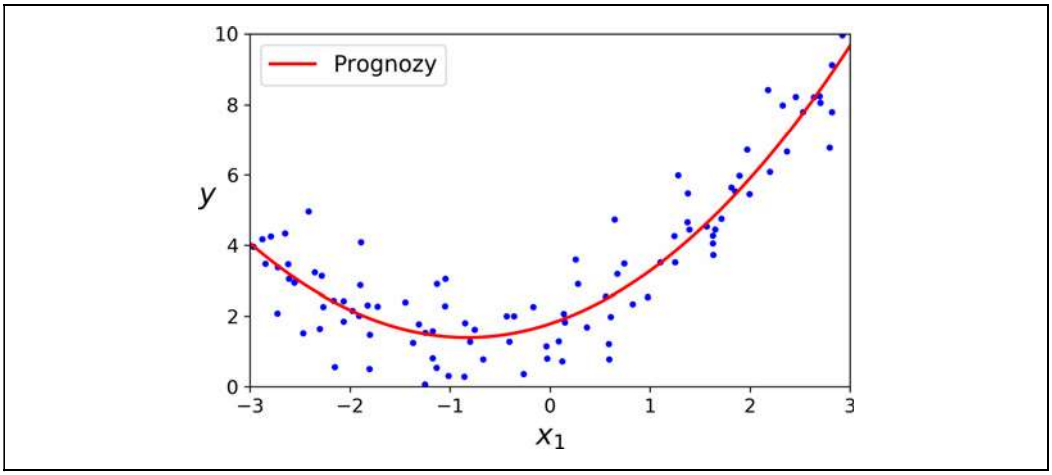
```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

[7] Równanie kwadratowe ma postać wzoru $y = ax^2+bx+c$.

Obiekt `X_poly` zawiera teraz oryginalną cechę X oraz jej kwadrat. Teraz możemy dopasować model `LinearRegression` do tak rozszerzonego zbioru danych uczących (rysunek 4.13):

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

Nieźle: model szacuje wzór $\hat{y} = 0{,}56x_1^2 + 0{,}93x_1 + 1{,}78$, podczas gdy oryginalna funkcja ma postać $y = 0{,}5x_1^2 + 1{,}0x_1 + 2{,}0 + szum\ Gaussowski$.



Rysunek 4.13. *Prognozy modelu regresji wielomianowej*

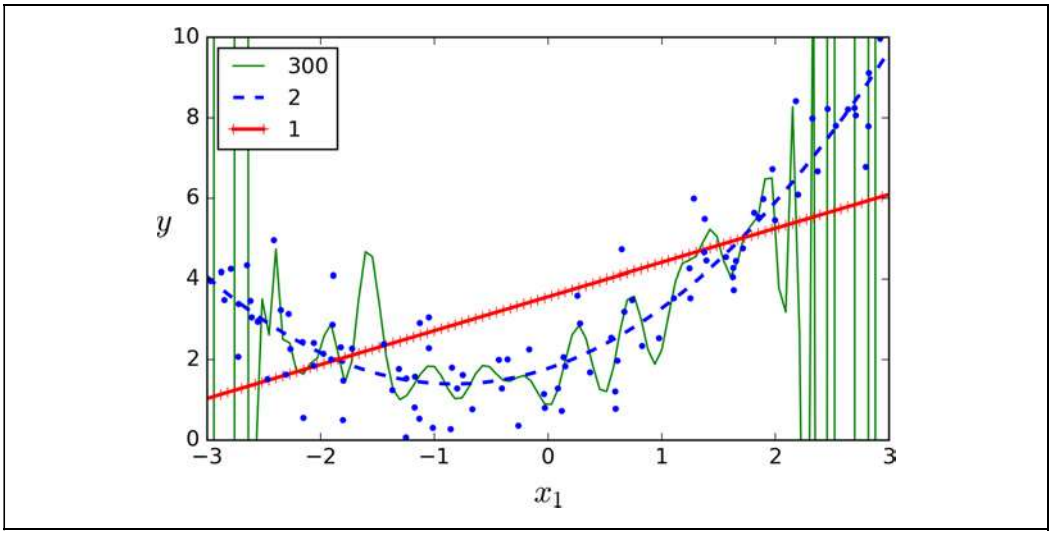
Zauważ, że w przypadku istnienia wielu cech model regresji wielomianowej jest w stanie znajdować powiązania pomiędzy nimi (czego nie jest w stanie dokonać klasyczny model regresji liniowej). Jest to możliwe, ponieważ klasa `PolynomialFeatures` dodaje również kombinacje cech aż do wyznaczonego stopnia. Przykładowo gdybyśmy mieli do czynienia z dwiema cechami, *a* i *b*, klasa `PolynomialFeatures` przy założonym parametrze `degree=3` dodałaby nie tylko cechy a^2, a^3, b^2 i b^3, lecz również ich kombinacji ab, a^2b i ab^2.



Klasa `PolynomialFeatures(degree=d)` przekształca każdą macierz zawierającą *n* cech w tablicę składającą się z $\frac{(n+d)!}{n!d!}$ cech, gdzie *n!* oznacza **silnię z n** (czyli $1{\times}2{\times}3{\times}\ldots{\times}n$). Wystrzegaj się kombinatorycznej eksplozji liczby cech!

Krzywe uczenia

Jeżeli przeprowadzasz regresję wielomianową wysokiego stopnia, najprawdopodobniej uda Ci się znacznie łatwiej dopasować model do danych niż za pomocą standardowej regresji liniowej. Na przykład na rysunku 4.14 widzimy efekt regresji liniowej trzysetnego stopnia użytej wobec wygenerowanego uprzednio zbioru danych uczących; porównujemy wyniki ze zwykłą regresją liniową oraz



Rysunek 4.14. Regresja wielomianowa wysokiego stopnia

modelem kwadratowym (wielomianowym drugiego stopnia). Zwróć uwagę, jak bardzo jest „rozbujany" wykres regresji trzeciego stopnia w celu zbliżenia się do jak największej liczby próbek uczących.

Taki model wysokiego stopnia w znacznym stopniu ulega przetrenowaniu, z kolei regresja liniowa — niedotrenowaniu. W tym przypadku najlepiej przeprowadza generalizację model kwadratowy, co ma sens, ponieważ zbiór danych został wygenerowany za pomocą równania kwadratowego. Zasadniczo jednak nie będziemy wiedzieć, jaki wzór został wykorzystany do utworzenia zestawu danych, w jaki więc sposób mamy określić złożoność modelu? Skąd mamy wiedzieć, czy dany model ulega przetrenowaniu lub niedotrenowaniu?

W rozdziale 2. korzystaliśmy z metody sprawdzianu krzyżowego do oszacowania wydajności uogólniania modelu. Jeżeli model sprawuje się dobrze wobec danych uczących, ale według wskaźników kroswalidacji nie radzi sobie z generalizowaniem, to znaczy, że ulega przetrenowaniu. W przypadku, gdy model nie radzi sobie zarówno ze zbiorem uczącym, jak i z uogólnianiem, jest niedotrenowany. Jest to jeden ze sposobów na stwierdzenie, czy model jest za prosty lub zbyt złożony.

Inną metodą określenia stopnia złożoności modelu jest analiza **krzywych uczenia** (ang. *learning curves*): są to krzywe wydajności modelu wobec zbioru uczącego i walidacyjnego w funkcji rozmiaru zbioru uczącego (lub przebiegu uczącego). Aby uzyskać taki wykres, wystarczy kilkakrotnie wytrenować model wobec podzbiorów zestawu uczącego cechujących się różnymi rozmiarami. W poniższym kodzie definiujemy funkcję generującą wykres krzywych uczenia modelu przy użyciu przykładowych danych uczących:

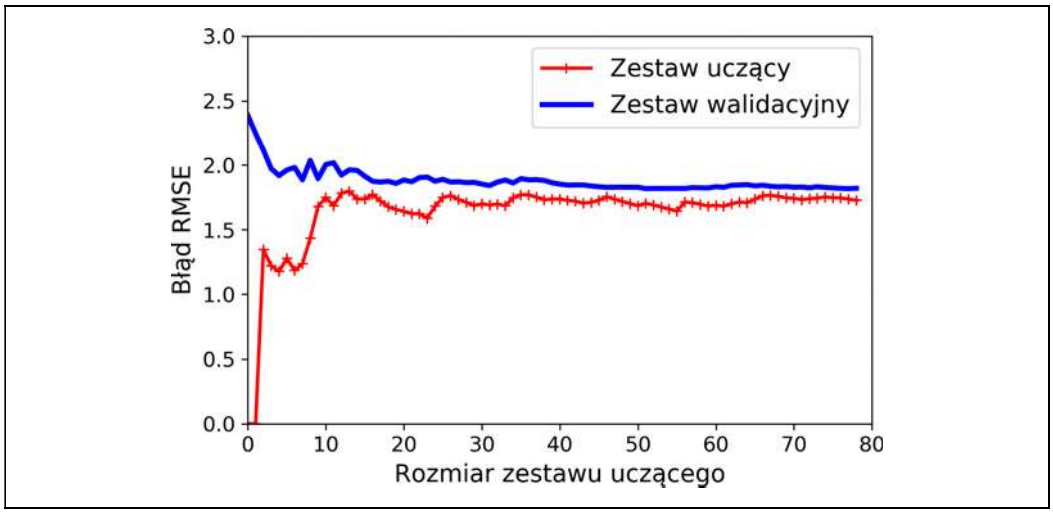
```
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

def plot_learning_curves(model, X, y):
   X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)
   train_errors, val_errors = [], []
```

```
    for m in range(1, len(X_train)):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(y_train[:m], y_train_predict))
        val_errors.append(mean_squared_error(y_val, y_val_predict))
    plt.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
```

Przyjrzyjmy się krzywym uczenia standardowego modelu regresji liniowej (rysunek 4.15):

```
lin_reg = LinearRegression()
plot_learning_curves(lin_reg, X, y)
```



Rysunek 4.15. Krzywe uczenia

Ten niedotrenowany model wymaga nieco dłuższego wyjaśnienia. Najpierw przyjrzyjmy się wydajności wobec zbioru danych uczących: gdy znajdują się w nim tylko jeden lub dwa przykłady, model jest w stanie wyuczyć się ich doskonale, dlatego wykres rozpoczyna się w punkcie 0. Jednak w miarę dodawania kolejnych przykładów zmniejsza się wydajność algorytmu, ponieważ dane stają się zaszumione i nieliniowe. Z tego powodu błąd dla zbioru uczącego pnie się w górę aż do osiągnięcia wypłaszczenia — od tej chwili dodawanie kolejnych próbek nie będzie ani zmniejszało, ani zwiększało średniego błędu. Przyjrzyjmy się teraz wydajności modelu wobec zbioru walidacyjnego. Jeżeli jest wykorzystywanych tylko kilka przykładów, model nie jest w stanie dobrze uogólniać wyników, przez co błąd walidacyjny jest na samym początku całkiem duży. Następnie wraz z pojawianiem się kolejnych przykładów następuje uczenie modelu, w wyniku czego wartość błędu stopniowo maleje. Jednak również w tym przypadku funkcja liniowa nie opisuje dobrze tych danych, dlatego ostatecznie poziom błędu staje się wypłaszczony, o wartości bardzo zbliżonej do błędu zbioru uczącego.

Te krzywe uczenia są charakterystyczne dla niedotrenowanego modelu. Obydwie osiągnęły wypłaszczenie; znajdują się blisko siebie i mają dość duże wartości.



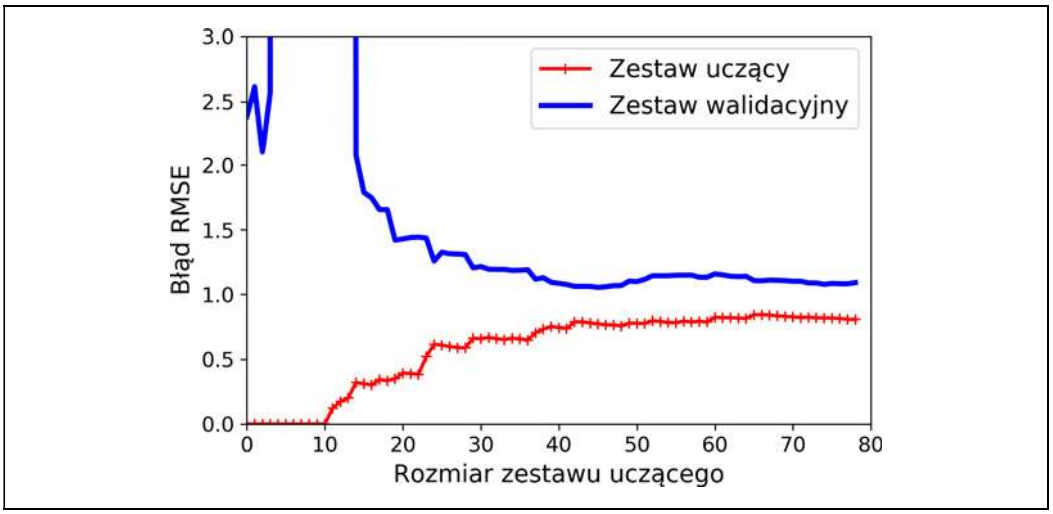
Jeżeli Twój model jest niedotrenowany, dodawanie kolejnych próbek uczących niewiele pomoże. Musisz skorzystać z bardziej złożonego modelu lub wprowadzić lepsze cechy.

Sprawdźmy teraz krzywe uczenia dziesięciostopniowego modelu wielomianowego wobec tych samych danych (rysunek 4.16):

```
from sklearn.pipeline import Pipeline

polynomial_regression = Pipeline([
     ("poly_features", PolynomialFeatures(degree=10, include_bias=False)),
     ("lin_reg", LinearRegression()),
   ])

plot_learning_curves(polynomial_regression, X, y)
```



Rysunek 4.16. *Krzywe uczenia dziesięciostopniowego modelu wielomianowego*

Wykresy te przypominają nieco krzywe uczenia modelu liniowego, możemy jednak dostrzec dwie bardzo istotne różnice:

- Błąd dla zbioru uczącego osiąga znacznie niższy poziom niż w przypadku modelu regresji liniowej.
- Istnieje przerwa pomiędzy obydwiema krzywymi. Oznacza to, że model radzi sobie znacznie lepiej z danymi uczącymi niż ze zbiorem walidacyjnym, co stanowi wyznacznik przetrenowania modelu. Gdybyśmy jednak korzystali ze znacznie większych zestawów danych, odległość pomiędzy krzywymi zmniejszałaby się stopniowo.



Jednym ze sposobów na poprawienie przetrenowanego modelu jest wprowadzenie dodatkowych przykładów aż do zrównania się błędu danych uczących z błędem danych walidacyjnych.

Kompromis pomiędzy obciążeniem a wariancją

Ważną konsekwencją rozważań łączących statystykę z uczeniem maszynowym jest fakt, że błąd generalizacji modelu można wyrazić w postaci sumy trzech odmiennych rodzajów błędów:

- **Obciążenie** (ang. *bias*): ta składowa błędu uogólnienia wynika z nieprawidłowych założeń, np. założenia, że dane są liniowe, podczas gdy w rzeczywistości są opisane funkcją kwadratową. Model o wysokiej wartości obciążenia ma największą szansę na niedotrenowanie wobec danych uczących[8].
- **Wariancja** (ang. *variance*): ta składowa wynika z nadmiernej czułości modelu na drobne wahania w wartościach danych uczących. Model cechujący się dużą liczbą stopni swobody (np. wielostopniowy model wielomianowy) prawdopodobnie będzie miał dużą wariancję, a przez to będzie przetrenowywany wobec danych uczących.
- **Błąd nieredukowalny** (ang. *irreducible error*): ta składowa stanowi konsekwencję zaszumienia danych. Jedynym sposobem pozbycia się jej jest oczyszczenie danych (np. naprawa źródeł danych takich jak zepsute czujniki lub wykrywanie i usuwanie elementów odstających).

Zwiększanie złożoności modelu zazwyczaj zwiększa jego wariancję i zmniejsza obciążenie. Odwrotny proces następuje w przypadku uproszczenia modelu. Dlatego mówimy tu u kompromisie.

Regularyzowane modele liniowe

Jak widzieliśmy w rozdziałach 1. i 2., dobrym sposobem zmniejszenia przetrenowania modelu jest jego regularyzacja (tzn. ograniczenie): im mniej stopni swobody, tym trudniej przetrenować model wobec danych. Prostym sposobem na regularyzację modelu wielomianowego jest zmniejszenie liczby stopni wielomianu.

W przypadku modelu liniowego regularyzacja jest przeważnie osiągana poprzez ograniczenie wag modelu. Przyjrzymy się teraz modelom regresji grzbietowej, regresji metodą LASSO i metodzie elastycznej siatki, w których stosowane są różne metody ograniczania wag.

Regresja grzbietowa

Regresja grzbietowa (ang. *ridge regression*, zwana także **regularyzacją Tichonowa** — ang. *Tikhonov regularization*) stanowi regularyzowaną odmianę regresji liniowej: do funkcji kosztu zostaje dodany **człon regularyzacyjny** o postaci $\alpha\sum_{i=1}^{n}\theta_i^2$ .W ten sposób model jest zmuszony nie tylko do dopasowywania się do danych, lecz także do utrzymywania jak najmniejszych wartości wag. Zwróć uwagę, że człon regularyzacyjny powinien zostać dodany do funkcji kosztu jedynie na etapie nauki. Po wytrenowaniu modelu chcemy ocenić wydajność modelu za pomocą nieregularyzowanego wskaźnika wydajności.

Za pomocą hiperparametru α określamy stopień regularyzacji modelu. Jeżeli $\alpha = 0$, to mamy do czynienia ze standardową regresją liniową. W przypadku bardzo dużych wartości hiperparametru α wartości

[8] Nie należy mylić błędu obciążenia z punktem obciążenia (ang. *bias term*) modeli liniowych.

wszystkich wag są bardzo bliskie zeru, w wyniku czego uzyskujemy prostą przebiegającą wzdłuż średniej zbioru danych. Równanie 4.8 przedstawia funkcję kosztu regresji grzbietowej[9].

Równanie 4.8. Funkcja kosztu regresji grzbietowej

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha\frac{1}{2}\sum_{i=1}^{n}\theta_i^2$$



Dość powszechnie się zdarza, że funkcja kosztu użyta w czasie uczenia różni się od pomiaru wydajności zastosowanego na etapie testowania. Powodem różnic może być nie tylko regularyzacja, lecz także fakt, że dobra funkcja kosztu użyta podczas uczenia powinna mieć nadające się do optymalizacji pochodne, natomiast wskaźnik wydajności stosowany podczas testowania powinna być jak najbliższy ostatecznemu celowi. Na przykład klasyfikatory są często trenowane za pomocą takiej funkcji kosztu jak logarytmiczna funkcja straty (do której powrócimy za chwilę), ale oceniane przy użyciu precyzji/pełności.

Zwróć uwagę, że punkt obciążenia θ_0 nie jest regularyzowany (suma rozpoczyna się od $i = 1$, nie $i = 0$). Jeśli zdefiniujemy $\boldsymbol{w}$ jako wektor wag cech (od θ_1 do θ_n), to człon regularyzacyjny będzie równy $\frac{1}{2}(\|\boldsymbol{w}\|_2)^2$, gdzie $\|\boldsymbol{w}\|_2$ oznacza normę ℓ_2 wektora wag[10]. W przypadku gradientu prostego po prostu dodajemy człon αw do wektora gradientów MSE (równanie 4.6).



Przed przeprowadzeniem regresji grzbietowej należy przeskalować dane (np. za pomocą klasy `StandardScaler`), gdyż metoda ta jest czuła na skalę cech wejściowych. Dotyczy to większości regularyzowanych modeli.

Rysunek 4.17 prezentuje kilka modeli regresji grzbietowej wyuczonych wobec danych liniowych; modele te różnią się wartościami hiperparametru α. Na lewym wykresie używamy standardowych modeli regresji grzbietowej, dzięki czemu uzyskujemy liniowe prognozy. Na prawym wykresie rozszerzamy najpierw dane za pomocą klasy `PolynomialFeatures(degree=10)`, następnie je skalujemy (`StandardScaler`), a na koniec stosujemy regresję grzbietową wobec otrzymanych cech: mamy tu do czynienia z regresją wielomianową z regularyzacją grzbietową. Zwróć uwagę, że wzrost wartości hiperparametru α prowadzi do uzyskania gładszych (mniej poszarpanych, rozsądniejszych) predykcji; zmniejszamy w ten sposób wariancję, ale zwiększamy obciążenie modelu.

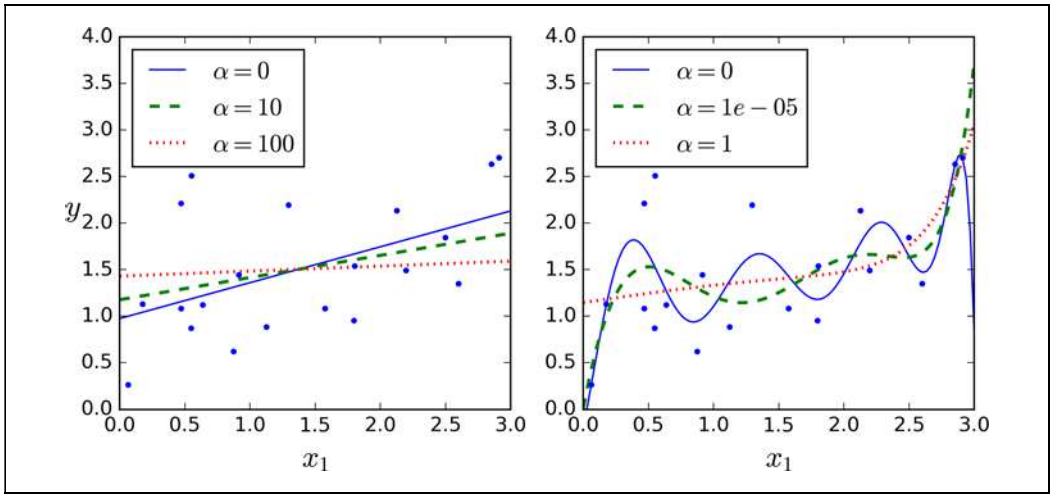
Podobnie jak w przypadku regresji liniowej, możemy przeprowadzać regularyzację grzbietową przy użyciu jawnego równania lub stosując metodę gradientu prostego. Również wady i zalety obydwu rozwiązań są takie same. Jawny wzór pokazuję w równaniu 4.9, gdzie $\boldsymbol{A}$ stanowi **macierz jednostkową**[11] o rozmiarze $(n+1)\times(n+1)$; wyjątek stanowi w niej wartość 0 umieszczona w lewej górnej komórce — reprezentuje ona punkt obciążenia.

---

[9] Funkcje kosztu o długich nazwach są często zapisywane jako $J(\theta)$; będziemy korzystać z tej notacji w dalszej części książki. Rodzaj omawianej funkcji kosztu z łatwością wywnioskujesz z kontekstu.

[10] Normy zostały omówione w rozdziale 2.

[11] Macierz kwadratowa wypełniona zerami oprócz komórek tworzących główną przekątną (od lewego górnego rogu do prawego dolnego), które zawierają wartość 1.



Rysunek 4.17. *Model liniowy (po lewej) i wielomianowy (po prawej) z wyznaczonymi różnymi stopniami regresji grzbietowej*

Równanie 4.9. Jawny wzór regresji grzbietowej

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^T\mathbf{X} + \alpha\mathbf{A}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

Poniżej prezentuję sposób przeprowadzania regresji grzbietowej w module Scikit-Learn za pomocą jawnego wzoru (odmiany równania 4.9, w której wykorzystujemy technikę faktoryzacji macierzy zaprojektowaną przez André-Louisa Cholesky'ego):

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([[1.55071465]])
```

A tu widzimy to samo, tym razem za pomocą stochastycznego spadku wzdłuż gradientu[12]:

```
>>> sgd_reg = SGDRegressor(penalty="l2")
>>> sgd_reg.fit(X, y.ravel())
>>> sgd_reg.predict([[1.5]])
array([ 1.47012588])
```

Hiperparametr `penalty` służy do wyznaczania rodzaju członu regularyzacyjnego. Wartość `l2` oznacza, że chcemy dodać do funkcji kosztu człon regularyzacyjny równy połowie kwadratu normy ℓ_2 wektora wag — czyli, krótko mówiąc, chcemy zastosować regresję grzbietową.

[12] Możesz ewentualnie zastosować klasę `Ridge` z mechanizmem rozwiązywania (ang. *solver*) `sag`, czyli algorytm stochastycznego uśrednionego gradientu, który stanowi odmianę standardowego algorytmu SGD. Więcej informacji znajdziesz w prezentacji „Minimizing Finite Sums with the Stochastic Average Gradient Algorithm" (*https://www.cs.ubc.ca/~schmidtm/Documents/2014_Google_SAG.pdf*) autorstwa Marka Schmidta i in. (Uniwersytet Kolumbii Brytyjskiej).

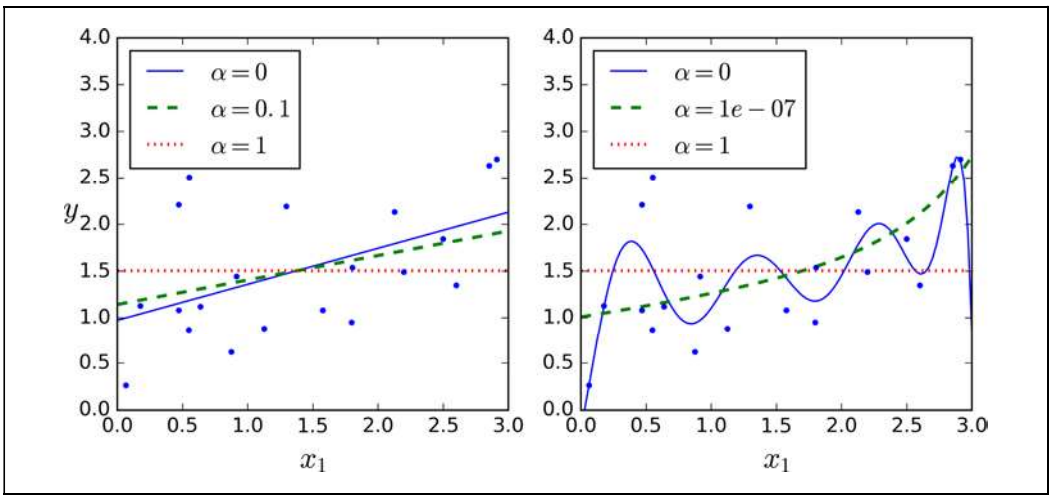
Regresja metodą LASSO

Regresja metodą LASSO (ang. *least absolute shrinkage and selection operator regression* —operator najmniejszej bezwzględnej redukcji i wyboru) stanowi kolejną regularyzowaną odmianę regresji liniowej; podobnie jak w regresji liniowej dodajemy tu człon regularyzacyjny do funkcji kosztu, w tym jednak przypadku korzystamy z normy ℓ_1 wektora wag, a nie z połowy kwadratu normy ℓ_2 (równanie 4.10).

Równanie 4.10. Funkcja kosztu regresji metodą LASSO

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha \sum_{i=1}^{n} |\theta_i|$$

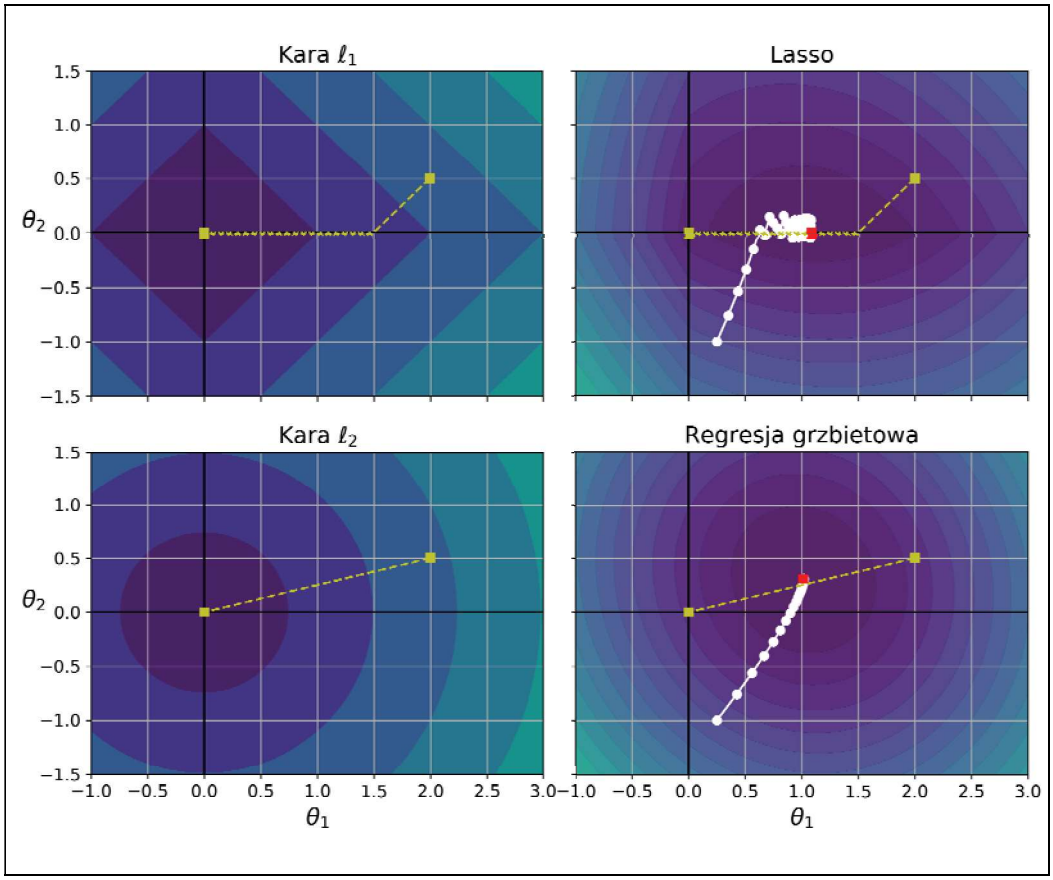
Na rysunku 4.18 wykorzystujemy te same dane, co na rysunku 4.17, ale zastępujemy modele regresji grzbietowej modelami regresji LASSO i wprowadzamy mniejsze wartości parametru α.



Rysunek 4.18. *Model liniowy (po lewej) i wielomianowy (po prawej) z wyznaczonymi różnymi stopniami regresji metodą LASSO*

Ważną własnością regresji metodą LASSO jest fakt, że dąży ona do całkowitej eliminacji wag w najmniej istotnych cechach (np. poprzez zmianę ich wartości na 0). Przykładowo, linia przerywana na prawym wykresie (rysunek 4.18; wykres o wartości $\alpha = 10^{-7}$) przypomina niemalże liniowy wykres funkcji kwadratowej: wszystkie wagi cech wielomianowych wyższego stopnia są równe zeru. Innymi słowy, regresja metodą LASSO automatycznie przeprowadza dobór cech i generuje **model rzadki** (tj. zawierający niewiele niezerowych wag cech).

Gdy spojrzysz na rysunek 4.19, zrozumiesz, dlaczego jest to takie ważne: osie reprezentują dwa parametry modelu, natomiast kontury tła ukazują różne funkcje straty. Na lewym górnym wykresie kontury tła symbolizują funkcję straty ℓ_1 ($|\theta_1|+|\theta_2|$), która maleje liniowo w miarę zbliżania się do dowolnej osi. Na przykład jeśli zainicjujesz model z parametrami $\theta_1 = 2$ i $\theta_2 = 0{,}5$, algorytm gradientu prostego będzie równomiernie zmniejszał obydwa parametry (co zostało pokazane za pomocą żółtej linii przerywanej), zatem parametr $\theta_2 = 0{,}5$ osiągnie wartość 0 jako pierwszy (ponieważ od początku znajdował się bliżej zera). Następnie algorytm będzie pędził, dopóki θ_1 nie osiągnie wartości 0 (algorytm



Rysunek 4.19. Porównanie regularyzacji grzbietowej i metody LASSO

będzie trochę „skakał", gdyż gradienty ℓ_1 nigdy nie zbliżają się do 0 — dla każdego parametru uzyskują one wartość –1 albo 1). Na prawym górnym wykresie kontury przedstawiają funkcję kosztu LASSO (tzn. funkcję kosztu MSE wraz z funkcją straty ℓ_1). Małe białe kółka pokazują ścieżkę gradientu prostego dążącą do optymalizowania niektórych parametrów modelu, które zostały zainicjowane z wartościami ok. $\theta_1 = 0{,}25$ i $\theta_2 = -1$ — zwróć uwagę, że również tutaj algorytm dociera szybko do $\theta_2 = 0$, a następnie znowu zaczyna „skakać" wokół optimum globalnego (symbolizowanego przez czerwony kwadrat). Gdybyśmy zwiększyli wartość parametru α, optimum globalne zostałoby przesunięte w lewo wraz z żółtą linią przerywaną, natomiast w przypadku zmniejszenia wartości α optimum globalne przemieściłoby się w prawo (w tym przykładzie optymalne parametry nieregularyzowanej funkcji MSE przyjmują wartości $\theta_1 = 2$ i $\theta_2 = 0{,}5$).

Dwa dolne wykresy przedstawiają to samo, ale regularyzację ℓ_1 zastępujemy karą ℓ_2. Na lewym dolnym wykresie widzimy, że funkcja straty ℓ_2 maleje w miarę oddalania się od środka układu współrzędnych, zatem algorytm gradientu prostego dąży wprost w kierunku tego punktu. Kontury widoczne na prawym dolnym wykresie reprezentują funkcję kosztu regresji grzbietowej (funkcję kosztu MSE wraz z funkcją straty ℓ_2). Występują tu dwie główne różnice w stosunku do metody

LASSO. Po pierwsze, gradienty maleją w miarę zbliżania się do optimum globalnego, zatem algorytm gradientu prostego zwalnia w naturalny sposób, co pomaga w uzyskaniu zbieżności (gdyż nie występuje zjawisko „skakania"). Po drugie, optymalne parametry (symbolizowane czerwonym kwadratem) stopniowo zbliżają się do środka układu współrzędnych w miarę zwiększania parametru α, ale nigdy nie zostają całkowicie wyeliminowane.



Aby w metodzie gradientu prostego uniknąć „skakania" wokół optimum pod koniec realizowania algorytmu LASSO, musisz stopniowo zmniejszać wartość współczynnika uczenia w trakcie uczenia modelu (nie wyeliminujemy całkowicie tego zjawiska, ale kolejne kroki będą coraz mniejsze aż do uzyskania zbieżności).

Funkcja kosztu LASSO nie jest różniczkowalna w punkcie $\theta_i = 0$ (dla $i = 1, 2, \ldots, n$), ale algorytm gradientu prostego będzie dobrze działał, jeśli użyjesz **wektora podgradientów** $\boldsymbol{g}$[13], gdy dowolny $\theta_i = 0$. Równanie 4.11 zawiera wzór wektora podgradientów, który możemy wykorzystać w algorytmie gradientu prostego wraz z funkcją kosztu LASSO.

Równanie 4.11. Wektor podgradientów w regresji metodą LASSO

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) + \alpha \begin{pmatrix} sign(\theta_1) \\ sign(\theta_2) \\ \vdots \\ sign(\theta_n) \end{pmatrix} \quad \text{gdzie } sign(\theta_i) = \begin{cases} -1 \text{ jesli } \theta_i < 0 \\ 0 \text{ jesli } \theta_i = 0 \\ +1 \text{ jesli } \theta_i > 0 \end{cases}$$

Poniżej prezentuję mały przykład użycia klasy `Lasso` w module Scikit-Learn. Zauważ, że możesz użyć zamiast tego klasy `SGDRegressor(penalty="l1")`.

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

Zauważ, że możesz użyć zamiast tego klasy `SGDRegressor(penalty="l1")`.

Metoda elastycznej siatki

Metoda elastycznej (ang. *elastic net*) siatki stanowi rozwiązanie pośrednie pomiędzy regresją grzbietową a metodą LASSO. Człon regularyzacyjny w tym przypadku tworzy prosta kombinacja członów obydwu wspomnianych technik, my zaś możemy nim sterować za pomocą współczynnika proporcji r. Gdy $r = 0$, metoda elastycznej siatki staje się równoważna regresji grzbietowej, natomiast przy $r = 1$ zachowuje się jak metoda LASSO (równanie 4.12).

[13] Możesz traktować wektor podgradientów w nieróżniczkowalnym punkcie jako wektor pośredni wektorów gradientów wokół tego punktu.

Równanie 4.12. Funkcja kosztu metody elastycznej siatki

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + r\alpha\sum_{i=1}^{n}\left|\theta_i\right| + \frac{1-r}{2}\alpha\sum_{i=1}^{n}\theta_i^2$$

Kiedy więc należy stosować regresję liniową (tj. nieregularyzowaną), a kiedy metodę grzbietową, LASSO albo elastyczną siatkę? Niemal zawsze warto wprowadzić chociaż odrobinę regularyzacji, dlatego staraj się unikać zwykłej regresji liniowej. Dobrym domyślnym rozwiązaniem okazuje się regresja grzbietowa, ale jeśli podejrzewasz, że będzie przydatnych tylko kilka cech, lepiej wybrać metodę LASSO lub elastycznej siatki, ponieważ, jak już wiemy, dążą one do zerowania wag najmniej użytecznych cech. Generalnie bardziej preferowana jest metoda elastycznej siatki, gdyż metoda LASSO zaczyna nieprawidłowo działać, gdy liczba cech przewyższa liczbę próbek uczących lub gdy istnieje silna korelacja pomiędzy kilkoma cechami.

Poniżej prezentuję przykład użycia klasy `ElasticNet` (parametr `l1_ratio` odpowiada współczynnikowi proporcji *r*):

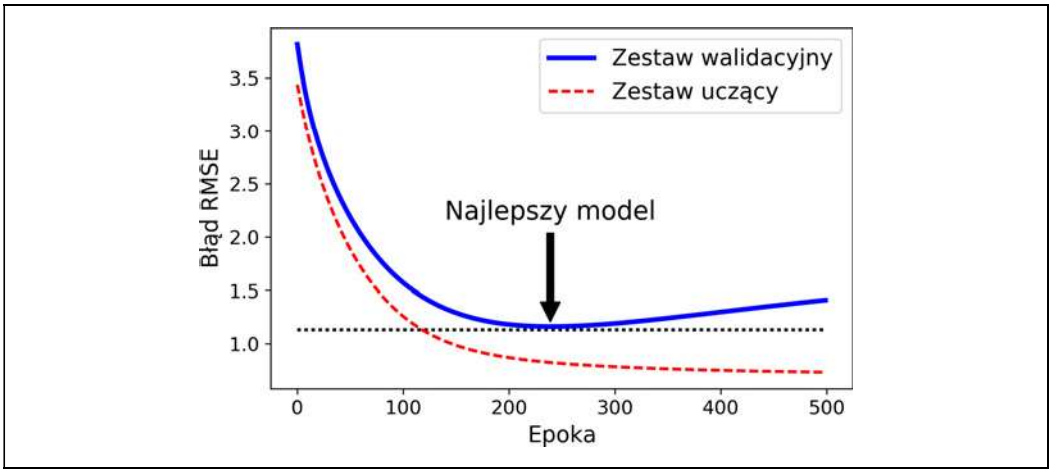
```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

Wczesne zatrzymywanie

Zupełnie innym mechanizmem regularyzacji iteracyjnych algorytmów uczących, takich jak metody gradientu prostego, jest zakończenie uczenia w momencie osiągnięcia minimum błędu walidacyjnego. Jest to tak zwane **wczesne zatrzymywanie** (ang. *early stopping*). Rysunek 4.20 przedstawia złożony model (w tym przypadku model regresji wielomianowej wysokiego stopnia), który jest trenowany za pomocą algorytmu wsadowego gradientu prostego. W miarę upływu kolejnych epok algorytm uczy się, a jego błąd predykcji (RMSE) wobec zestawu uczącego maleje w naturalny sposób, wraz z błędem prognozowania wobec danych walidacyjnych. Jednak po pewnym czasie błąd prognozowania przestaje maleć i zaczyna znów rosnąć. Jest to wyraźny znak, że model zaczyna ulegać przetrenowaniu. Dzięki wczesnemu zatrzymywaniu przerywamy proces uczenia w momencie osiągnięcia minimalnej wartości błędu predykcji. Jest to tak prosta i skuteczna technika regularyzacji, że Geoffrey Hinton określił ją jako „piękny, darmowy obiad".



W przypadku metod stochastycznego spadku wzdłuż gradientu i schodzenia po gradiencie z minigrupami generowane krzywe nie są takie gładkie i może być trudno określić, czy algorytm osiągnął minimum. Jedno z rozwiązań polega na zatrzymaniu procesu uczenia, gdy zauważymy, że błąd walidacyjny już od pewnego czasu biegnie w pobliżu wartości minimalnej (gdy będziemy pewni, że model nie będzie już lepiej działał), a następnie wprowadzeniu parametrów modelu występujących w momencie osiągnięcia minimalnej wartości błędu.



Rysunek 4.20. *Regularyzacja metodą wczesnego zatrzymywania*

Tak wygląda implementacja metody wczesnego zatrzymywania:

```
from sklearn.base import clone

# Przygotowuje dane
poly_scaler = Pipeline([
        ("poly_features", PolynomialFeatures(degree=90, include_bias=False)),
        ("std_scaler", StandardScaler())
    ])
X_train_poly_scaled = poly_scaler.fit_transform(X_train)
X_val_poly_scaled = poly_scaler.transform(X_val)
sgd_reg = SGDRegressor(max_iter=1, tol=-np.infty, warm_start=True,
                       penalty=None, learning_rate="constant", eta0=0.0005)

minimum_val_error = float("inf")
best_epoch = None
best_model = None
for epoch in range(1000):
    sgd_reg.fit(X_train_poly_scaled, y_train) # Kontynuuje z miejsca, w którym zostało przerwane działanie
    y_val_predict = sgd_reg.predict(X_val_poly_scaled)
    val_error = mean_squared_error(y_val, y_val_predict)
    if val_error < minimum_val_error:
        minimum_val_error = val_error
        best_epoch = epoch
        best_model = clone(sgd_reg)
```

Zwróć uwagę, że dzięki parametrowi `warm_start=True` po wywołaniu metody `fit()` model będzie kontynuował uczenie od miejsca, w którym zostało przerwane.

Regresja logistyczna

W rozdziale 1. wyjaśniłem, że pewne algorytmy regresyjne można stosować w zadaniach klasyfikujących (i odwrotnie). **Regresja logistyczna** (ang. *logistic regression*, zwana także **regresją logitową** — ang. *logit regression*) służy powszechnie do szacowania prawdopodobieństwa przynależności przykładu do określonej klasy (np. jakie jest prawdopodobieństwo, że dana wiadomość jest

spamem?). Jeżeli oszacowane prawdopodobieństwo przekracza 50%, to model prognozuje, że próbka ta należy do tej klasy (zwanej klasą pozytywną; jest ona oznaczona etykietą *1*), w przeciwnym razie stwierdza, że przykład nie stanowi części określonej klasy (tzn. że przynależy do klasy negatywnej, oznaczanej etykietą *0*). Model ten wykazuje więc cechy klasyfikatora binarnego.

Szacowanie prawdopodobieństwa

Jaki jest więc mechanizm działania modelu regresji logistycznej? Podobnie jak w przypadku regresji liniowej, w modelu regresji logistycznej wyliczamy ważoną sumę cech wejściowych (wraz z punktem obciążenia), nie są jednak wyświetlane bezpośrednie wyniki, lecz **funkcja logistyczna** rezultatu (równanie 4.13).

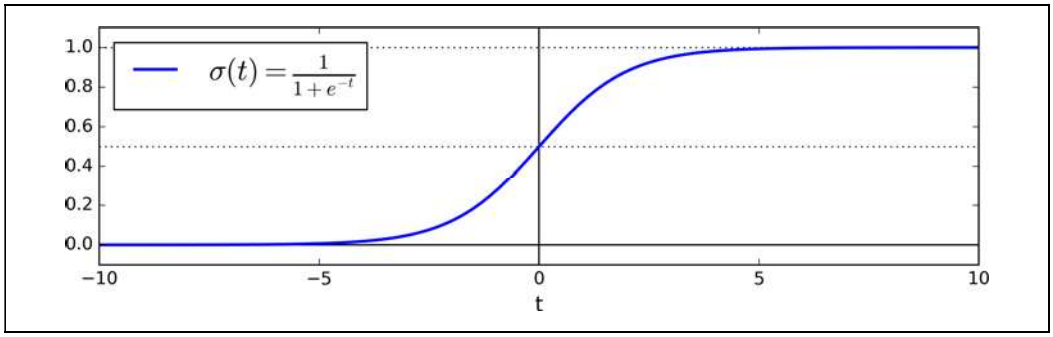
Równanie 4.13. Szacowane prawdopodobieństwo w modelu regresji logistycznej (postać wektorowa)

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\left(\mathbf{x}^T\boldsymbol{\theta}\right)$$

Funkcja logistyczna zapisywana w postaci $\sigma(\cdot)$ stanowi **funkcję sigmoidalną** (tzn. jej przebieg przypomina literę S), której wartości mieszczą się w przedziale pomiędzy 0 i 1. Została ona zaprezentowana w równaniu 4.14 i na rysunku 4.21.

Równanie 4.14. Funkcja logistyczna

$$\sigma(t) = \frac{1}{1+\exp(-t)}$$



Rysunek 4.21. *Wykres funkcji logistycznej*

Po oszacowaniu prawdopodobieństwa $\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x})$ przynależności przykładu $\boldsymbol{x}$ do pozytywnej klasy model regresji logistycznej z łatwością może obliczyć prognozę $\hat{y}$ (równanie 4.15).

Równanie 4.15. Prognoza modelu regresji logistycznej

$$\hat{y} = \begin{cases} 0 \text{ jeśli } \hat{p} < 0,5 \\ 1 \text{ jeśli } \hat{p} \geq 0,5 \end{cases}$$

Zwróć uwagę, że $\sigma(t) < 0,5$, gdy $t < 0$, a $\sigma(t) \geq 0,5$, gdy $t \geq 0$, zatem model regresji logistycznej przewiduje 1, jeżeli wynik $\mathbf{x}^T\boldsymbol{\theta}$ jest dodatni, a 0, jeżeli jest ujemny.



Wynik *t* często jest nazywany logitem. Nazwa wywodzi się z faktu, że funkcja logitowa, zdefiniowana jako *logit*(*p*) = *log*(*p*/(*1*–*p*)), stanowi odwrotność funkcji logistycznej. Rzeczywiście, jeśli obliczysz logit szacowanego prawdopodobieństwa *p*, okaże się, że wynikiem jest *t*. Możesz spotkać się również z nazwą **logarytm szans** (ang. *log odds*), gdyż jest to logarytm ilorazu szacowanego prawdopodobieństwa klasy pozytywnej i szacowanego prawdopodobieństwa klasy negatywnej.

Funkcje ucząca i kosztu

Wiesz już, w jaki sposób model regresji logistycznej oszacowuje prawdopodobieństwa i wylicza prognozy. Jak jednak możemy go uczyć? Celem uczenia jest dobranie takiego wektora parametrów $\boldsymbol{\theta}$, żeby model szacował wysokie prawdopodobieństwo dla pozytywnych próbek ($y = 1$), a niskie dla przykładów negatywnych ($y = 0$). Koncepcja ta znajduje odzwierciedlenie w funkcji kosztu dla pojedynczego przykładu uczącego $\boldsymbol{x}$ (równanie 4.16).

Równanie 4.16. Funkcja kosztu dla pojedynczej próbki uczącej

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{jeśli } y = 1 \\ -\log(1-\hat{p}) & \text{jeśli } y = 0 \end{cases}$$

Taka funkcja kosztu ma sens, ponieważ –*log*(*t*) osiąga bardzo duże wartości, gdy *t* dąży do zera, zatem koszt będzie duży, jeśli model oszacuje prawdopodobieństwo bliskie zeru dla klasy pozytywnej; będzie on osiągać również znaczne wartości, gdy wartość prawdopodobieństwa będzie bliska 1 dla klasy negatywnej. Z drugiej strony, funkcja –*log*(*t*) będzie dążyła do zera, gdy *t* będzie zbliżało się do 1, więc koszt będzie bliski zeru, jeśli oszacowane prawdopodobieństwo przynależności do klasy negatywnej będzie niemal zerowe lub bardzo bliskie 1 dla klasy pozytywnej, a przecież dokładnie o to nam chodzi.

Funkcja kosztu dla całego zbioru uczącego stanowi średni koszt wyliczony dla każdego przykładu w zbiorze. Możemy to zapisać w postaci pojedynczego wzoru, zwanego **logarytmiczną funkcją straty** (ang. *log loss*), zaprezentowanego w równaniu 4.17.

Równanie 4.17. Funkcja kosztu regresji logistycznej (logarytmiczna funkcja straty)

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{p}^{(i)}\right)\right]$$

Zła wiadomość jest taka, że nie znamy jawnego wzoru pozwalającego na obliczanie wartości θ minimalizującej tę funkcję kosztu (nie istnieje odpowiednik równania normalnego). Dobra wieść jest taka, że funkcja ta jest wypukła, zatem metodą gradientu prostego (lub dowolnym innym algorytmem optymalizacji) z pewnością znajdziemy minimum lokalne (pod warunkiem że współczynnik uczenia nie jest za duży, a my mamy odpowiednio dużo czasu). Równanie 4.18 ukazuje pochodne cząstkowe funkcji kosztu dla j-tego parametru θ_j.

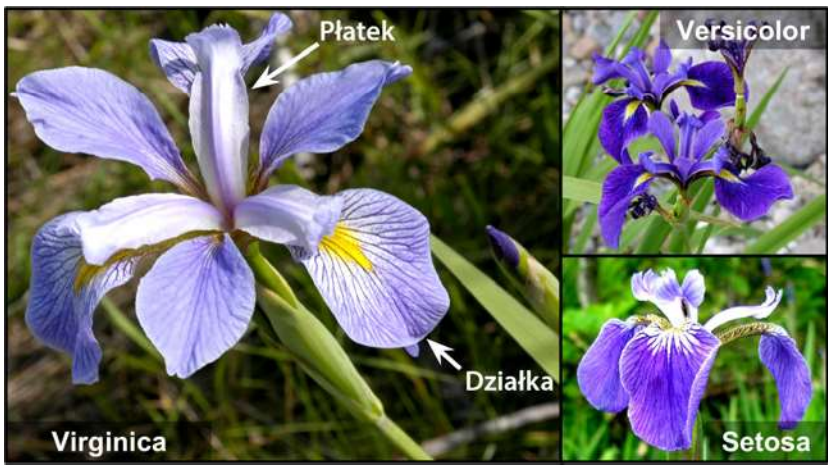
Równanie 4.18. Pochodne cząstkowe logistycznej funkcji kosztu

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

Wzór ten bardzo przypomina równanie 4.5: dla każdej próbki jest wyliczany błąd predykcji, który zostaje następnie pomnożony przez wartość j-tej cechy, po czym następuje wyliczenie wartości średniej ze wszystkich przykładów uczących. Po uzyskaniu wektora gradientów zawierającego wszystkie pochodne cząstkowe możemy go użyć w algorytmie wsadowego gradientu prostego. I to by było na tyle — wiesz już, jak wyuczyć model regresji logistycznej. W przypadku metody stochastycznego spadku wzdłuż gradientu pobieralibyśmy po jednej próbce, a w metodzie schodzenia po gradiencie z minigrupami korzystalibyśmy z niewielkich podzbiorów uczących.

Granice decyzyjne

Model regresji logistycznej zilustrujemy na przykładzie zbioru danych *Iris*. Jest to słynny zbiór danych zawierający parametry (długość i szerokość płatków oraz działek kielicha) 150 kwiatów z trzech gatunków kosaćca: *Iris setosa*, *Iris versicolor* i *Iris virginica* (rysunek 4.22).



Rysunek 4.22. *Kwiaty trzech gatunków kosaćca*[14]

Spróbujmy stworzyć klasyfikator rozpoznający gatunek *Iris virginica* jedynie na podstawie szerokości płatka. Najpierw wczytajmy dane:

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> list(iris.keys())
['data', 'target', 'target_names', 'DESCR', 'feature_names', 'target']
>>> X = iris["data"][:, 3:] # Szerokość płatka
>>> y = (iris["target"] == 2).astype(np.int) # 1, jeśli wykryje Iris virginica, w przeciwnym razie 0
```

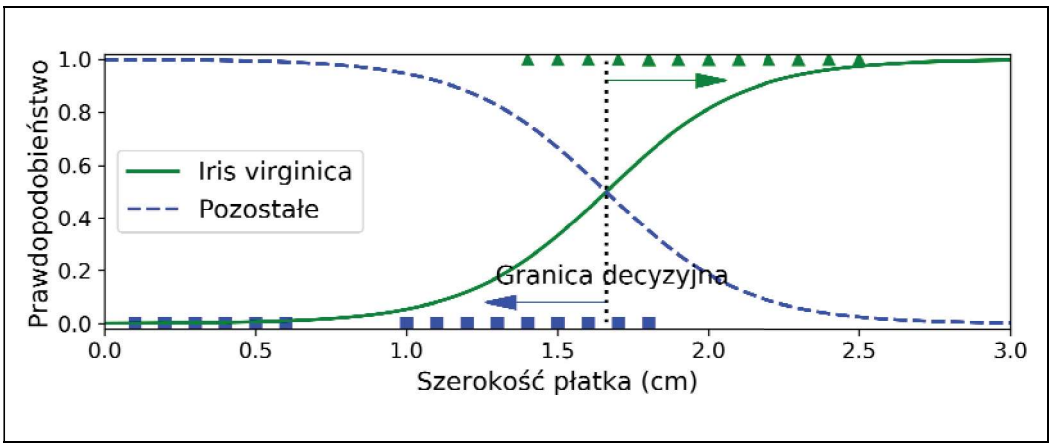
[14] Zdjęcia pochodzą ze stron serwisu Wikipedia poświęconych poszczególnym gatunkom omawianej rośliny. Fotografia kwiatu *Iris virginica* została wykonana przez Franka Mayfielda (licencja Uznanie autorstwa — na tych samych warunkach 2.0, *https://creativecommons.org/licenses/by-sa/2.0/deed.pl*), autorem zdjęcia przedstawiającego *Iris versicolor* jest D. Gordon E. Robertson (Uznanie autorstwa — na tych samych warunkach 3.0, *https://creativecommons.org/licenses/by-sa/3.0/deed.pl*), natomiast zdjęcie *Iris setosa* stanowi własność publiczną.

Wytrenujmy teraz model regresji logistycznej:

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression()
log_reg.fit(X, y)
```

Sprawdźmy oszacowane prawdopodobieństwa przynależności do wyznaczonego gatunku dla kwiatów o szerokości płatków w przedziale od 0 do 3 cm (rysunek 4.23)[15]:



Rysunek 4.23. *Oszacowane prawdopodobieństwa i granica decyzyjna*

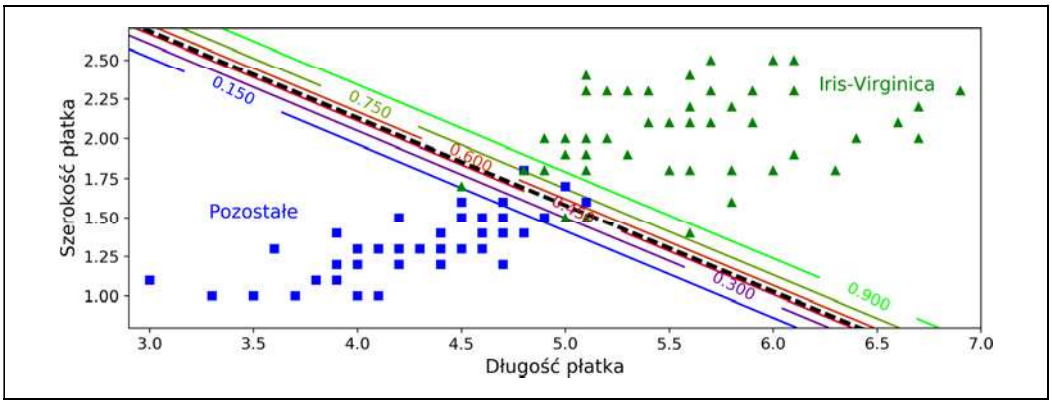
```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:, 1], "g-", label="Iris virginica")
plt.plot(X_new, y_proba[:, 0], "b--", label="Pozostałe")
# Dodatkowo nie pokazany w książce kod modułu Matplotlib, dzięki któremu wykres będzie ładnie wyglądał
```

Szerokość płatków w kwiatach *Iris virginica* (reprezentowana przez trójkąty) mieści się w zakresie pomiędzy 1,4 a 2,5 cm, natomiast jest ona zazwyczaj mniejsza u pozostałych gatunków kosaćca (symbolizowanych przez kwadraty) — w granicach od 0,1 do 1,8 cm. Jak widać, szerokości płatków we wszystkich gatunkach nakładają się w pewnym zakresie. W przypadku szerokości powyżej 2 cm klasyfikator jest całkiem pewny, że kwiat przynależy do gatunku *Iris virginica* (uzyskujemy duże prawdopodobieństwo przynależności do klasy *Iris virginica*), natomiast w przypadku węższych płatków (poniżej 1 cm) klasyfikator z dużą pewnością uznaje, że dany kwiat nie należy do gatunku *Iris virginica* (duże prawdopodobieństwo przynależności do klasy *Pozostałe*). W przypadku wartości środkowych algorytm klasyfikujący nie jest już taki pewny. Jeśli jednak chcemy przewidzieć klasę (za pomocą metody `predict()` zamiast `predict_proba()`), zostanie zwrócona wartość najbardziej prawdopodobnej klasy. Dlatego w okolicy szerokości 1,6 cm istnieje **granica decyzyjna** wyznaczająca prawdopodobieństwo 50% przynależności do którejś z dwóch klas: jeżeli szerokość płatka przekracza 1,6 cm, klasyfikator uzna, że kwiat należy do gatunku *Iris virginica*, w przeciwnym wypadku zaklasyfikuje go do grupy pozostałych gatunków (nawet jeśli nie będzie całkowicie tego pewny):

[15] Funkcja `reshape()` z modułu NumPy pozwala wyznaczyć jeden wymiar o wartości –1, co oznacza „niezdefiniowany”: wartość zostaje wywnioskowana z długości tablicy i pozostałych wymiarów.

```
>>> log_reg.predict([[1.7], [1.5]])
array([1, 0])
```

Na rysunku 4.24 widzimy ten sam zbiór danych, tym razem jednak korzystamy z dwóch cech: szerokości i długości płatka. Klasyfikator regresji logistycznej po wytrenowaniu jest w stanie przy użyciu tych dwóch cech szacować prawdopodobieństwo przynależności nowego przykładu do klasy *Iris-Virginica*. Linia przerywana pokazuje punkty, w których klasyfikator oszacowuje 50% prawdopodobieństwa: jest to granica decyzyjna modelu. Zauważmy, że granica ta jest liniowa[16]. Równoległe do niej linie określają punkty wskazujące określone prawdopodobieństwo, począwszy od 15% (lewa dolna część wykresu) aż do 90% (prawa górna część). Zgodnie z tym modelem wszystkie kwiaty znajdujące się po prawej górnej stronie od zielonej linii mają ponad 90% szans na przynależność do gatunku *Iris virginica*.



Rysunek 4.24. *Liniowa granica decyzyjna*

Podobnie jak w przypadku poprzednich modeli liniowych, jesteśmy w stanie regularyzować algorytmy regresji logistycznej za pomocą kar ℓ_1 lub ℓ_2. Istotnie, moduł Scikit-Learn domyślnie stosuje tu regularyzację ℓ_2.



W modelu `LogisticRegression` za siłę regularyzacji odpowiada nie hiperparametr `alpha` (występujący w innych modelach liniowych), lecz jego odwrotność: `C`. Im większa jego wartość, tym *mniejszy* stopień regularyzacji modelu.

Regresja softmax

Możemy uogólnić model regresji logistycznej do bezpośredniej obsługi wielu klas, bez konieczności uczenia i łączenia wielu klasyfikatorów binarnych (proces ten został omówiony w rozdziale 3.). Tego typu model nosi nazwę **regresji softmax** lub **wielomianowej/wielorakiej regresji logistycznej** (ang. *multinomial logistic regression*).

Jej koncepcja jest prosta: dla danej próbki $\boldsymbol{x}$ model regresji softmax najpierw oblicza wynik $s_k(\boldsymbol{x})$ dla każdej klasy k, następnie oszacowuje prawdopodobieństwo przynależności do danej klasy, stosując wobec

[16] Jest to zbiór punktów x określanych funkcją $\theta_0+\theta_1 x_1+\theta_2 x_2 = 0$, której wykres ma przebieg liniowy.

tych wyników **funkcję typu softmax** (zwaną także **znormalizowaną funkcją wykładniczą**). Wzór służący do wyliczania wyników $s_k(\boldsymbol{x})$ powinien wyglądać znajomo, gdyż jest tożsamy ze wzorem na prognozy regresji liniowej (równanie 4.19).

Równanie 4.19. Wynik funkcji softmax dla klasy k

$$s_k\left(\mathbf{x}\right) = \mathbf{x}^T\boldsymbol{\theta}^{(k)}$$

Zwróć uwagę, że każda klasa ma własny, wyspecjalizowany wektor parametrów $\boldsymbol{\theta}^{(k)}$. Wszystkie te wektory są zazwyczaj przechowywane w rzędach **macierzy parametrów** $\boldsymbol{\Theta}$.

Po wyliczeniu wyników każdej klasy dla przykładu $\boldsymbol{x}$ możemy oszacować prawdopodobieństwo $\hat{p}_k$ przynależności tej próbki do klasy k, przekazując funkcji softmax (równanie 4.20) te wyniki. Dla każdego zostaje wyliczona jego eksponenta, po czym zostaje przeprowadzona normalizacja (otrzymany rezultat dzielimy przez sumę wszystkich eksponent). Wyniki są zazwyczaj nazywane logitami lub logarytmami szans (chociaż w rzeczywistości są one nienormalizowanymi logarytmami szans).

Równanie 4.20. Funkcja softmax

$$\hat{p}_k = \sigma\left(\mathbf{s}\left(\mathbf{x}\right)\right)_k = \frac{\exp\left(s_k\left(\mathbf{x}\right)\right)}{\sum_{j=1}^{K}\exp\left(s_j\left(\mathbf{x}\right)\right)}$$

W tym równaniu:

- K — liczba klas,
- $\boldsymbol{s}(\boldsymbol{x})$ — wektor zawierający wyniki każdej klasy dla przykładu $\boldsymbol{x}$,
- $\sigma(\boldsymbol{s}(\boldsymbol{x}))_k$ — szacowane prawdopodobieństwo przynależności próbki $\boldsymbol{x}$ do klasy k przy znanych wynikach wszystkich klas dla tego przykładu.

Podobnie jak w przypadku klasyfikatora regresji logistycznej model regresji softmax prognozuje klasę o najwyższym oszacowanym prawdopodobieństwie (czyli klasę o największym wyniku), o czym możemy przekonać się, spoglądając na równanie 4.21.

Równanie 4.21. Prognoza klasyfikatora regresji Softmax

$$\hat{y} = \arg\max_k \sigma\left(\mathbf{s}\left(\mathbf{x}\right)\right)_k = \arg\max_k s_k\left(\mathbf{x}\right) = \arg\max_k\left(\left(\boldsymbol{\theta}^{(k)}\right)^T \cdot \mathbf{x}\right)$$

Operator *argmax* zwraca wartość zmiennej maksymalizującej funkcję. W tym równaniu zwraca ona wartość parametru k maksymalizującą szacowane prawdopodobieństwo $\sigma(\boldsymbol{s}(\boldsymbol{x}))_k$.



Klasyfikator regresji softmax prognozuje tylko jedną klasę naraz (tzn. jest modelem wieloklasowym, nie wielowyjściowym), dlatego należy go używać wyłącznie ze wzajemnie wykluczającymi się klasami, takimi jak różne gatunki roślin. Nie możemy stosować go do rozpoznawania wielu osób na jednym zdjęciu.

Skoro wiemy już, w jaki sposób model oszacowuje prawdopodobieństwa i przeprowadza prognozy, możemy zabrać się za jego uczenie. Celem trenowania jest tu uzyskanie modelu oszacowującego wysokie prawdopodobieństwo dla klasy docelowej (a zatem niskie prawdopodobieństwo dla pozosta-

łych klas). Minimalizowanie widocznej w równaniu 4.22 funkcji kosztu (zwanej **entropią krzyżową**) powinno nam w tym pomóc, ponieważ w ten sposób model jest karany w przypadku szacowania niewielkiego prawdopodobieństwa dla klasy docelowej. Entropia krzyżowa jest często wykorzystywana do sprawdzania, w jakim stopniu zbiór oszacowanych prawdopodobieństw zgadza się z klasami docelowymi.

Równanie 4.22. Funkcja kosztu — entropia krzyżowa

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

W tym równaniu:

- $y_k^{(i)}$ jest docelowym prawdopodobieństwem przynależności *i*-tego przykładu do klasy *k*. Zasadniczo przyjmuje ono wartość 1 lub 0, w zależności od tego, czy dany przykład przynależy do klasy *k*.

Zwróć uwagę, że jeżeli dostępne są tylko dwie klasy ($K = 2$), ta funkcja kosztu przekształca się w funkcję kosztu regresji logistycznej (logarytmiczną funkcję straty) (równanie 4.17).

Entropia krzyżowa

Pojęcie entropii krzyżowej (ang. *cross enthropy*) wywodzi się z teorii informacji. Załóżmy, że chcesz codziennie skutecznie przesyłać informacje o pogodzie. Jeżeli mamy osiem możliwości (słonecznie, deszczowo itd.), możemy zakodować każdą z nich za pomocą trzech bitów, ponieważ $2^3 = 8$. Jeżeli jednak uważasz, że niemal codziennie będzie słoneczna pogoda, znacznie wydajniejsze byłoby zakodowanie stanu „słonecznie" w postaci tylko jednego bitu (0), a pozostałych siedmiu opcji w formie czterobitowej (pierwszy bit miałby wartość 1). Entropia krzyżowa mierzy uśrednioną liczbę bitów, jaką rzeczywiście przeznaczasz na każdą możliwość. Jeżeli Twoje założenia dotyczące pogody są niezawodne, wartość entropii krzyżowej będzie równa entropii samej pogody (np. jej naturalnej nieprzewidywalności). Jeśli jednak Twoje założenia są nieprawidłowe (np. często pada), wartość entropii krzyżowej będzie większa o czynnik zwany **dywergencją Kullbacka-Leiblera** (KL).

Entropia krzyżowa pomiędzy dwoma rozkładami prawdopodobieństwa *p* i *q* jest definiowana wzorem $H(p,q) = -\sum_x p(x)\log q(x)$ (przynajmniej wtedy, gdy obydwa rozkłady są dyskretne). Więcej informacji znajdziesz w nagranym przeze mnie filmie poświęconym temu zagadnieniu (*https://www.youtube.com/watch?v=ErfnhcEV1O8*).

Wektor gradientów dla tej funkcji kosztu w odniesieniu do $\boldsymbol{\theta}^{(k)}$ został ukazany w równaniu 4.23.

Równanie 4.23. Wektor gradientów entropii krzyżowej dla klasy k

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\mathbf{\Theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)}$$

Możemy teraz obliczyć wektor gradientów dla każdej klasy, a następnie użyć metody gradientu prostego (lub dowolnego innego algorytmu optymalizacji) do wyszukania macierzy parametrów $\boldsymbol{\Theta}$ minimalizującej funkcję kosztu.

Użyjmy regresji softmax do rozdzielenia kwiatów kosaćca na trzy klasy. Klasa `LogisticRegression` korzysta domyślnie ze strategii OvR podczas trenowania modelu przy użyciu przynajmniej trzech klas, mimo to możemy wyznaczyć wartość `multinomial` hiperparametru `multi_class` w celu przejścia na metodę regresji softmax. Musimy także zdefiniować mechanizm rozwiązywania obsługujący ten typ regresji, np. `solver="lbgfs"` (więcej szczegółów znajdziesz w dokumentacji modułu Scikit-Learn). Domyślnie jest również wykorzystywana regularyzacja ℓ_2, którą regulujemy przy użyciu hiperparametru `C`.

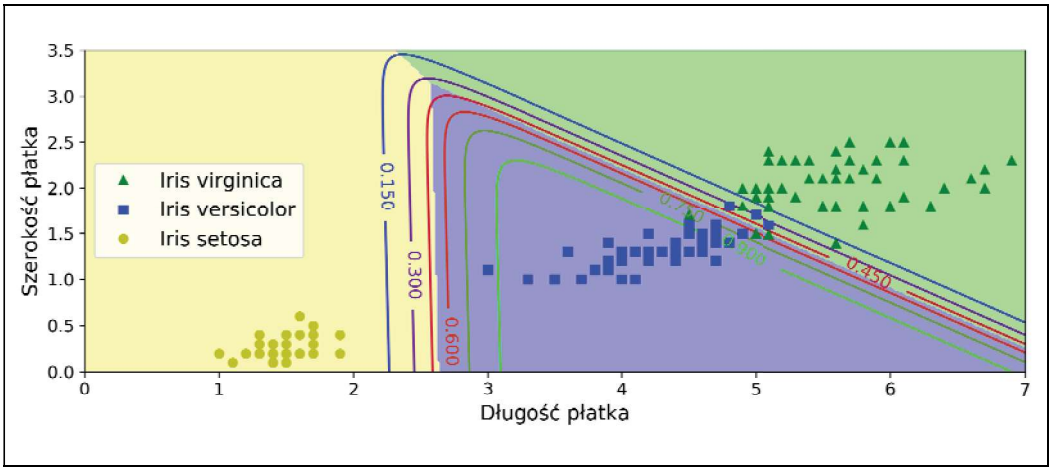
```
X = iris["data"][:, (2, 3)] # Długość płatka, szerokość płatka
y = iris["target"]

softmax_reg = LogisticRegression(multi_class="multinomial",solver="lbfgs", C=10)
softmax_reg.fit(X, y)
```

Teraz, gdy znajdziesz na polu kwiat kosaćca o płatkach długich na 5 cm i szerokich na 2 cm, możesz zapytać swój model, na jaki gatunek trafiłaś/trafiłeś, a uzyskasz odpowiedź, że na 94,2% jest to *Iris virginica* (klasa 2.), ewentualnie *Iris versicolor* (ale prawdopodobieństwo wynosi zaledwie 5,8%):

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]])
array([[6.38014896e-07, 5.74929995e-02, 9.42506362e-01]])
```

Na rysunku 4.25 widzimy uzyskane granice decyzyjne, symbolizowane przez różne kolory tła. Zwróć uwagę, że są one liniowe pomiędzy dwiema poszczególnymi klasami. Widzimy ponadto na wykresie prawdopodobieństwa przynależności do klasy *Iris versicolor*, ukazane za pomocą linii (np. linia oznaczona cyfrą *0.450* symbolizuje granicę decyzyjną wyznaczającą 45% przynależności do tejże klasy). Zwróć uwagę, że model może prognozować klasę, dla której zostało oszacowane prawdopodobieństwo mniejsze niż 50%. Przykładowo, w punkcie styku wszystkich trzech granic decyzyjnych prawdopodobieństwo przynależności do którejś klasy wynosi dla każdej z nich po 33%.



Rysunek 4.25. *Granice decyzyjne w regresji softmax*

Ćwiczenia

1. Którego modelu uczącego regresji liniowej należy użyć, jeżeli zbiór danych uczących zawiera miliony cech?
2. Załóżmy, że cechy w zbiorze danych uczących mieszczą się w różnych skalach. Którym algorytmom może to zaszkodzić i w jaki sposób? Jak możemy temu zaradzić?
3. Czy podczas uczenia modelu regresji logistycznej algorytm gradientu prostego może utknąć w minimum lokalnym?
4. Czy wszystkie metody gradientu prostego prowadzą do tego samego modelu, jeżeli damy im wystarczająco wiele czasu?
5. Załóżmy, że korzystamy z algorytmu wsadowego gradientu prostego i stworzyliśmy wykres błędu walidacyjnego w funkcji epoki. Jeżeli wartość tego błędu stale rośnie, co to może oznaczać? Jak można rozwiązać ten problem?
6. Czy zatrzymanie algorytmu schodzenia po gradiencie z minigrupami natychmiast po wykryciu wzrostu wartości błędu walidacyjnego jest dobrym pomysłem?
7. Który z omówionych algorytmów gradientu prostego najszybciej osiągnie optymalne rozwiązanie? Który z nich rzeczywiście uzyska zbieżność? W jaki sposób możesz doprowadzić pozostałe do zbieżności?
8. Załóżmy, że korzystamy z regresji wielomianowej. Rysujemy wykres krzywych uczenia i zauważamy dużą przerwę pomiędzy wykresem błędu dla danych uczących a wykresem błędu dla danych walidacyjnych. Co się dzieje? Na jakie trzy sposoby możesz rozwiązać ten problem?
9. Załóżmy, że używamy metody regresji grzbietowej i zauważamy, że błędy danych uczących i walidacyjnych są niemal równe i mają dużą wartość. Czy taki model cechuje się dużą wariancją, czy obciążeniem? Należy w takim wypadku zwiększyć wartość parametru α czy ją zmniejszyć?
10. Dlaczego należy używać:
    a. algorytmu regresji grzbietowej zamiast standardowej regresji liniowej (tzn. nieregularyzowanej)?
    b. algorytmu regresji typu LASSO zamiast regresji grzbietowej?
    c. metody elastycznej siatki zamiast regresji typu LASSO?
11. Załóżmy, że chcesz klasyfikować zdjęcia wykonywane w pomieszczeniach/na zewnątrz oraz dzienne/nocne. Należałoby w tym celu zaimplementować dwa klasyfikatory regresji logistycznej czy jeden klasyfikator typu softmax?
12. Zaimplementuj algorytm wsadowego gradientu prostego wykorzystujący mechanizm wczesnego zatrzymywania w modelu regresji softmax (bez korzystania z modułu Scikit-Learn).

Rozwiązanie tych zadań znajdziesz w dodatku A.